

Responsible AI, Security, and Evaluation

This guide covers the responsible-AI lifecycle from data collection to deployment governance. Each chapter is self-contained and interview-focused. Symbols used repeatedly are: dataset $D = \{(x_i, y_i, a_i)\}_{i=1}^n$, features x , target y , protected attribute a , prediction $\hat{y}$, model f , and loss ℓ .

Bias in Data

1. Overview

Data bias is a systematic mismatch between the data used to build or evaluate a model and the population, process, or decision the model is meant to represent. It can enter through sampling, measurement, labeling, historical decisions, missingness, or deployment feedback. Because models learn statistical regularities rather than social intent, biased data can produce inaccurate, discriminatory, or unsafe outcomes in hiring, lending, healthcare, vision, NLP, recommenders, and LLMs.

2. Intuition

A survey taken only at a technology conference cannot reliably describe an entire city. More training rows do not fix the problem if every new row comes from the same skewed source. Data bias is therefore mainly a *coverage and data-generating-process* problem, not merely a dataset-size problem.

3. Prerequisites

- Sampling and conditional probability
- Descriptive statistics and confidence intervals
- Train/validation/test splits and distribution shift
- Classification metrics and confusion matrices
- Basic Pandas and scikit-learn

4. Core Concepts

Subtopic	Meaning and why it matters	Example	Interview angle
Selection bias	Inclusion probability depends on the subject or outcome	Credit data contains only past approved applicants	Why cannot more IID data remove it?
Representation bias	Important groups or regions are underrepresented	Few dark-skin images in a vision dataset	Audit group counts and uncertainty
Measurement bias	A proxy measures groups differently	Healthcare cost used as a proxy for medical need	Validate construct, instrument, and proxy

Subtopic	Meaning and why it matters	Example	Interview angle
Label bias	Labels reflect annotator prejudice or inconsistent policy	"Culture fit" ratings in hiring	Measure inter-annotator agreement
Historical bias	Accurate records encode inequitable past decisions	Arrest data reflects policing patterns	Accuracy can reproduce injustice
Survivorship bias	Failed or missing cases disappear	Training on companies that survived	Model the selection mechanism
Aggregation bias	One model assumes the same relationship for all groups	Symptoms vary by demographic group	Consider interactions or calibrated subgroup models
Feedback-loop bias	Predictions influence future observations	Predictive policing sends more patrols, producing more recorded incidents	Separate observation from underlying prevalence

5. Algorithm / Working Process

1. Define the intended population, outcome, unit of analysis, and protected groups.
2. Draw a data-lineage map: source, inclusion rule, instruments, annotators, transformations, and missing rows.
3. Compare dataset group proportions and feature/label distributions with trustworthy population references.
4. Measure missingness, label agreement, proxy validity, subgroup sample size, and temporal/geographic coverage.
5. Train a baseline and report performance and calibration per group, including confidence intervals.
6. Mitigate at the source where possible: recollect, relabel, stratify, reweight, or revise the target.
7. Re-evaluate after deployment because user behavior and feedback loops change the distribution.

6. Mathematical Foundation

If the deployment distribution is $p_T(x, y)$ but training uses $p_S(x, y)$, empirical risk estimates target the wrong expectation:

$$R_T(f) = \mathbb{E}_{(x,y) \sim p_T}[\ell(f(x), y)] \neq R_S(f).$$

Under covariate shift, $p_T(y | x) = p_S(y | x)$, importance weighting estimates target risk:

$$R_T(f) = \mathbb{E}_{p_S} \left[\frac{p_T(x)}{p_S(x)} \ell(f(x), y) \right].$$

Representation can be summarized by $r_g = n_g/n$; compare it with the target-population rate π_g . A standardized mean difference for a numeric feature is

$$\text{SMD} = \frac{\bar{x}_1 - \bar{x}_0}{\sqrt{(s_1^2 + s_0^2)/2}}.$$

SMD measures distributional imbalance, not unfairness by itself. For labels, Cohen's $\kappa = (p_o - p_e)/(1 - p_e)$ discounts agreement expected by chance.

7. Practical Implementation

```
import pandas as pd
from sklearn.metrics import accuracy_score, recall_score

def bias_audit(df, group="group", label="y", pred="y_pred"):
    rows = []
    for value, part in df.groupby(group, dropna=False):
        rows.append({
            "group": value,
            "n": len(part),
            "share": len(part) / len(df),
            "label_rate": part[label].mean(),
            "accuracy": accuracy_score(part[label], part[pred]),
            "recall": recall_score(part[label], part[pred], zero_division=0),
        })
    return pd.DataFrame(rows).sort_values("group")

data = pd.DataFrame({
    "group": ["A"] * 6 + ["B"] * 4,
    "y":      [1, 1, 0, 1, 0, 1, 1, 1, 0, 1],
    "y_pred": [1, 0, 0, 1, 0, 1, 0, 0, 0, 1],
})
print(bias_audit(data))
```

8. Code Explanation

The audit groups rows by the sensitive or operational slice, then reports support, representation share, observed label rate, accuracy, and recall. `zero_division=0` makes an empty-positive slice explicit rather than crashing. In production, add confidence intervals, missingness, calibration, intersections such as gender $\times$ region, and comparison with a documented reference population.

9. Training / Evaluation

Split by the deployment unit—often time, patient, customer, or geography—before preprocessing. Preserve a final untouched test set. Report macro metrics and per-group confusion matrices, calibration, worst-group performance, and bootstrap confidence intervals. Reweighting can improve target representation but creates high-variance estimates when weights are extreme. Do not balance the test set unless the reported estimand explicitly assumes a balanced population.

10. Complexity and Cost

Count-based audits take $O(nd)$ time for n rows and d audited fields. Intersectional analysis can grow exponentially with the number of attributes and needs larger samples. Recollection and expert relabeling usually dominate compute cost; standard audits are CPU-friendly.

11. Common Use Cases

- Auditing hiring, lending, insurance, and admissions datasets
- Checking medical-device coverage across hospitals and demographics
- Measuring language, dialect, culture, and toxicity-label coverage for LLMs
- Detecting geographic and sensor bias in autonomous systems
- Monitoring recommendation feedback loops

12. Common Mistakes

- Treating protected-group balance as proof of unbiased data
- Using model predictions as ground-truth labels
- Randomly splitting data with person, time, or site leakage
- Ignoring intersectional groups and small-sample uncertainty
- Removing a protected column while leaving perfect proxies
- Reweighting without checking extreme weights or target prevalence
- Confusing different base rates with either proof or absence of discrimination

13. Edge Cases / Limitations

The correct target population may be unknown or changing. Protected attributes may be missing, legally restricted, noisy, non-binary, or context-dependent. Small groups make estimates unstable; broad groups hide within-group variation. Some historical outcomes are fundamentally unsuitable targets, so no statistical correction can repair the construct.

14. Variations

- **Pre-processing mitigation:** resampling, reweighting, relabeling; placement-important and model-agnostic.
- **In-processing mitigation:** fairness constraints or adversarial debiasing; useful when training can change.
- **Post-processing mitigation:** group-aware thresholds; useful when only decisions can change, but may face legal constraints.
- **Causal bias analysis:** models pathways and counterfactuals; research-important when correlation is insufficient.
- **Active data collection:** acquires cases from poorly covered regions; valuable in production and CV.

15. Related Topics

Bias concerns the data-generating process; fairness defines normative constraints on outcomes. Dataset cards document discovered limitations. Privacy can conflict with subgroup auditing because sensitive attributes are needed to measure disparities. Domain shift, missing-data mechanisms, causal inference, and robust optimization provide technical tools for diagnosis and mitigation.

16. Interview Questions

1. **What is data bias?** A systematic distortion causing data to misrepresent the intended population, construct, or process.

2. **Bias versus variance?** Data/social bias is systematic mismatch; statistical model bias is approximation error; variance is sensitivity to samples.
3. **Can a balanced dataset still be biased?** Yes; labels, measurements, histories, and within-group coverage can remain biased.
4. **Why not delete protected attributes?** Proxies retain information, deletion prevents auditing, and unequal relationships may remain.
5. **How do you detect representation bias?** Compare group and intersection counts/distributions against the intended population with uncertainty.
6. **What is label bias?** Systematic label error caused by annotators, policies, proxies, or unequal observation.
7. **How would you mitigate selection bias?** Improve sampling first; otherwise model inclusion probabilities and use weighting/sensitivity analysis.
8. **What is a feedback loop?** Model decisions affect the data later used to retrain the model.
9. **Why report confidence intervals?** Small subgroup gaps may be noise; intervals expose uncertainty.
10. **What is the best first audit question?** "Who or what could not enter this dataset, and why?"

17. Practice Tasks

- Code group representation, missingness, label rates, confusion matrices, and bootstrap intervals.
- Audit Adult Income or COMPAS while documenting why the labels and population are imperfect.
- Compare random, temporal, and geographic splits.
- Debug an apparently fair model whose smallest intersection has zero recall.
- Extend the audit with importance weights and weight-clipping sensitivity.

18. Project Ideas

Project	What it does	Stack and dataset	Resume value
Hiring Data Auditor	Generates slice, proxy, and label-quality reports	Pandas, sklearn; Adult or synthetic hiring data	Data lineage and responsible ML
Vision Coverage Map	Finds demographic/lighting performance gaps	PyTorch, FairFace or BDD100K	CV evaluation and slicing
Recommendation Loop Simulator	Simulates exposure-driven retraining bias	NumPy, Streamlit; synthetic clicks	Causal/product thinking

19. Quick Revision

- **Key idea:** data can systematically misrepresent population or construct.
- **Main formula:** target risk uses p_T , sometimes estimated by importance weighting.
- **When to use:** before training and continuously after deployment.
- **Metrics:** support, SMD, missingness, agreement, group performance, calibration.
- **Trap:** "balanced groups" does not mean unbiased labels.
- **One-liner:** A model can faithfully learn biased data and still be wrong for the intended decision.

20. Final Cheat Sheet

Item	Summary
Definition	Systematic distortion in sampling, measurement, labels, history, or feedback
Input/output	Dataset + intended population → documented gaps and mitigation plan
Steps	Scope → lineage → slice → compare → mitigate → monitor
Hyperparameters	Group definitions, minimum support, weight clipping, alert thresholds
Metrics	Representation, SMD, agreement, per-group error/calibration
Pros/cons	Audits expose risks; they depend on valid reference data and group definitions
Best use	High-impact datasets and any system serving heterogeneous populations

Fairness

1. Overview

ML fairness studies how benefits, errors, and decision opportunities are distributed among individuals and groups. It combines technical metrics with legal, ethical, and domain judgment. A fairness objective is never chosen from mathematics alone: hiring, credit, healthcare, and content moderation attach different meanings and harms to false positives, false negatives, access, and long-term effects.

2. Intuition

“Treat everyone the same” and “produce equally good outcomes” can conflict. Giving every runner the same shoe size is procedurally identical but not useful. Fairness asks what should be equal—selection rates, error rates, calibration, treatment, or opportunity—and for whom.

3. Prerequisites

- Confusion matrix, precision, recall, false-positive rate, calibration
- Conditional probability and base rates
- Threshold classifiers and constrained optimization
- Protected attributes and intersectionality
- Basic causal and policy reasoning

4. Core Concepts

Concept	Meaning and importance	Simple example	Interview angle
Demographic parity	Equal positive-decision rates across groups	Equal interview-selection rate	Can hide qualification differences and label bias
Equal opportunity	Equal true-positive rates	Qualified candidates advance equally	Focuses on benefit of correct positives
Equalized odds	Equal TPR and FPR	Loan errors equal across groups	Stronger than equal opportunity

Concept	Meaning and importance	Simple example	Interview angle
Predictive parity	Equal precision	Approval means same repayment likelihood	Conflicts with equalized odds under unequal base rates
Calibration within groups	Same score means same outcome frequency	Score 0.8 implies ~80% outcome in each group	Score-level property, not threshold fairness
Individual fairness	Similar people receive similar outcomes	Near-identical applications treated alike	Requires a defensible similarity metric
Counterfactual fairness	Prediction unchanged if protected attribute changed in a causal counterfactual	Same person in a counterfactual gender world	Requires causal assumptions
Intersectional fairness	Audits combinations of identities	Gender × disability × region	Avoids fairness gerrymandering

5. Algorithm / Working Process

1. Identify stakeholders, decisions, protected groups, benefits, and harms.
2. Decide the construct and fairness criterion with domain/legal owners.
3. Establish a valid evaluation set and compute overall, group, and intersectional metrics.
4. Quantify gaps with uncertainty and inspect thresholds/calibration.
5. Mitigate through data changes, constrained training, or decision policies.
6. Evaluate utility–fairness trade-offs and secondary harms.
7. Document the chosen definition, rejected alternatives, owners, and monitoring triggers.

6. Mathematical Foundation

For protected attribute A :

$$\text{DP gap} = P(\hat{Y} = 1 \mid A = 1) - P(\hat{Y} = 1 \mid A = 0).$$

Equal opportunity requires $P(\hat{Y} = 1 \mid Y = 1, A = a)$ to be equal across a . Equalized odds additionally equalizes $P(\hat{Y} = 1 \mid Y = 0, A = a)$. Disparate impact is the selection-rate ratio

$$\text{DI} = \frac{P(\hat{Y} = 1 \mid A = 1)}{P(\hat{Y} = 1 \mid A = 0)}.$$

A constrained learner can solve

$$\min_{\theta} \frac{1}{n} \sum_i \ell(f_{\theta}(x_i), y_i) \quad \text{s.t.} \quad |g(\theta)| \leq \epsilon,$$

where g is a fairness gap and ϵ is an accepted tolerance. When base rates differ and prediction is imperfect, calibration, predictive parity, and equalized odds generally cannot all hold simultaneously; this is a policy trade-off, not a tuning bug.

7. Practical Implementation

```
import pandas as pd
from sklearn.metrics import confusion_matrix

def fairness_report(y, pred, group):
    out = []
    frame = pd.DataFrame({"y": y, "pred": pred, "group": group})
    for g, d in frame.groupby("group"):
        tn, fp, fn, tp = confusion_matrix(d.y, d.pred, labels=[0, 1]).ravel()
        out.append({
            "group": g, "n": len(d), "selection_rate": d.pred.mean(),
            "tpr": tp / (tp + fn) if tp + fn else float("nan"),
            "fpr": fp / (fp + tn) if fp + tn else float("nan"),
            "precision": tp / (tp + fp) if tp + fp else float("nan"),
        })
    report = pd.DataFrame(out).set_index("group")
    gaps = report.max(numeric_only=True) - report.min(numeric_only=True)
    return report, gaps
```

8. Code Explanation

The function builds the confusion matrix independently for every group and derives selection rate, TPR, FPR, and precision. Explicit denominator checks prevent misleading divisions. The max-min summary is useful for alerts, but production reports should show which groups form each gap, confidence intervals, thresholds, base rates, and intersectional slices.

9. Training / Evaluation

Choose fairness metrics before model selection. Use a deployment-representative holdout and prevent entity/time leakage. Tune model and threshold on validation data, never the final test. Track utility plus fairness: AUROC/PR-AUC, expected cost, calibration error, group TPR/FPR/precision, and worst-group loss. Bootstrap by the independent unit. Evaluate interventions for distribution shift and delayed outcomes.

10. Complexity and Cost

Metric computation is $O(n)$. Threshold search is approximately $O(n \log n)$ for sorting scores. Constrained or adversarial training adds optimization cost and can reduce utility. The harder cost is collecting protected attributes, stakeholder review, and monitoring long-delayed outcomes.

11. Common Use Cases

- Hiring screening and interview allocation
- Credit approval, pricing, and collections
- Medical diagnosis and resource prioritization
- Face recognition and biometric verification
- Content moderation and speech/toxicity classification
- Recommender exposure and marketplace ranking

12. Common Mistakes

- Choosing a convenient metric without defining harm
- Treating the 80% disparate-impact heuristic as universal proof
- Comparing point estimates for tiny groups without uncertainty
- Optimizing demographic parity when labels represent legitimate need—or biased history—without analysis
- Hiding group-specific failures behind macro averages
- Adjusting thresholds on the test set
- Assuming fairness through unawareness
- Ignoring accessibility, recourse, and long-term feedback

13. Edge Cases / Limitations

Fairness definitions can be mutually incompatible. Binary group labels erase identity and intersectionality. Ground truth can be delayed or biased. Group-specific thresholds may be prohibited or socially unacceptable in some jurisdictions. Static metrics miss strategic behavior, feedback loops, distribution shift, and harms outside the model boundary.

14. Variations

- **Pre-processing:** reweighting/resampling; simplest when data access exists.
- **In-processing:** reductions, regularizers, adversarial debiasing; project/research relevant.
- **Post-processing:** threshold optimization; practical for frozen models.
- **Distributionally robust optimization:** minimizes worst-group risk; important when group labels are reliable.
- **Causal/counterfactual fairness:** distinguishes allowed and disallowed causal paths; advanced research/interview topic.
- **Ranking fairness:** controls exposure rather than binary decisions; important for recommenders.

15. Related Topics

Bias diagnoses the origin of disparities; fairness specifies desired decision properties. Calibration concerns probability reliability, while equalized odds concerns errors. Causal inference is needed for counterfactual questions. Privacy may restrict collecting the very attributes needed for auditing. AI governance assigns approval and accountability for the normative choice.

16. Interview Questions

1. **What is ML fairness?** The study and governance of how model benefits, burdens, and errors are distributed.
2. **Demographic parity versus equal opportunity?** The former equalizes selection rates; the latter equalizes TPR among actual positives.
3. **Equal opportunity versus equalized odds?** Equalized odds also requires equal FPR.
4. **Why can fairness metrics conflict?** Unequal base rates plus imperfect prediction makes several conditional equalities mathematically incompatible.
5. **What is fairness through unawareness?** Removing protected attributes; it fails because proxies and structural differences remain.

6. **What is individual fairness?** Similar individuals should receive similar predictions under a task-appropriate similarity metric.
7. **How choose a fairness metric?** Start from stakeholders, harm, decision semantics, law, and intervention—not from library availability.
8. **Can group thresholds help?** Yes, they can meet error constraints, but require legal and ethical review.
9. **Why audit intersections?** Marginal parity can conceal severe harm to a subgroup combination.
10. **How monitor fairness?** Track group support, outcomes, errors, calibration, drift, complaints, and delayed labels with owned alerts.

17. Practice Tasks

- Implement DP, equal-opportunity, equalized-odds, and precision gaps from scratch.
- Train a classifier on Adult Income and plot accuracy–fairness trade-offs across thresholds.
- Bootstrap group metric confidence intervals.
- Debug Simpson's paradox between aggregate and intersectional results.
- Add a constrained optimization or reweighting experiment and document the policy assumption.

18. Project Ideas

Project	What it does	Stack and dataset	Resume value
Fair Loan Dashboard	Compares utility, calibration, and group gaps across thresholds	sklearn, Fairlearn, Streamlit; German Credit	End-to-end responsible ML
Ranking Exposure Auditor	Measures group exposure in top-k recommendations	Python, implicit; MovieLens + synthetic groups	Recommender fairness
Clinical Equity Monitor	Tracks site/demographic sensitivity with uncertainty	PyTorch, Evidently; MIMIC-derived task	High-stakes monitoring

19. Quick Revision

- **Key idea:** decide which benefits or errors should be comparable across groups.
- **Formula:** equal opportunity compares $P(\hat{Y} = 1 \mid Y = 1, A = a)$.
- **Use:** consequential decisions affecting heterogeneous populations.
- **Metrics:** selection, TPR/FPR, precision, calibration, worst-group loss.
- **Trap:** no single universal fairness metric.
- **One-liner:** Fairness is a socio-technical requirement expressed and tested with metrics, not a metric chosen in isolation.

20. Final Cheat Sheet

Item	Summary
Definition	Distribution of benefits, burdens, opportunities, and model errors
Input/output	Labels, scores, groups, policy → gaps and decision rule
Steps	Define harm → choose criterion → slice → mitigate → monitor

Item	Summary
Hyperparameters	Constraint tolerance, threshold, group granularity, min support
Metrics	DP, DI, TPR/FPR gaps, precision, calibration, worst-group risk
Pros/cons	Makes trade-offs measurable; definitions conflict and labels may be biased
Best use	Hiring, lending, health, ranking, moderation, biometrics

Explainability

1. Overview

Explainability comprises methods that communicate why a model produced an output or how it behaves. Explanations may be local or global, intrinsic or post-hoc, model-specific or model-agnostic. They support debugging, user recourse, scientific insight, audit, incident response, and regulatory documentation, but an attractive explanation is not automatically faithful to the model.

2. Intuition

An explanation is like a mechanic's diagnostic report: it selects evidence relevant to a question. "Why was this loan denied?", "What generally drives defaults?", and "What would change the decision?" require different explanations even for the same model.

3. Prerequisites

- Features, predictions, gradients, and probability scores
- Linear models, trees, and neural networks
- Conditional versus marginal dependence
- Basic optimization and sampling
- Familiarity with SHAP, LIME, counterfactuals, and saliency

4. Core Concepts

Concept	Meaning and importance	Example	Interview angle
Local explanation	Explains one prediction	Income and debt drove this denial	Not necessarily globally representative
Global explanation	Summarizes model behavior	Default risk rises with debt ratio	Aggregation can hide interactions
Feature attribution	Allocates output contribution to inputs	Token-level sentiment scores	Correlated features complicate credit
Counterfactual	Small feasible change producing another outcome	"Reduce utilization below 40%"	Must be actionable and causal enough

Concept	Meaning and importance	Example	Interview angle
Example-based	Uses similar/prototypical cases	Nearest approved applications	Similarity metric matters
Surrogate model	Approximates a complex model with a simple one	Small decision tree approximates a forest	Report fidelity
Saliency	Uses gradients/activations to highlight influential pixels/tokens	Heatmap on lesion region	Can be noisy and unfaithful
Explanation audience	Tailors content to developer, affected user, auditor, or regulator	Debug trace versus recourse statement	Explanations are question-dependent

5. Algorithm / Working Process

1. Specify audience and question: cause, sensitivity, recourse, debugging, or summary.
2. Select method compatible with modality and model access.
3. Establish background/reference data and meaningful feature representation.
4. Generate local explanations over representative and high-risk slices.
5. Test fidelity, stability, sensitivity, sparsity, and human usefulness.
6. Compare methods and run sanity checks such as parameter/label randomization.
7. Present model output, uncertainty, limitations, and explanation scope together.

6. Mathematical Foundation

A local additive attribution model often has

$$g(z') = \phi_0 + \sum_{j=1}^M \phi_j z'_j,$$

where z'_j indicates the presence of an interpretable feature. A counterfactual solves

$$x' = \arg \min_z \lambda L(f(z), y_{target}) + d(z, x) + \Omega(z)$$

subject to feasibility constraints. d keeps the change small and Ω encourages sparse/actionable changes. Explanation fidelity for a surrogate can be measured as agreement or R^2 between $g(x)$ and $f(x)$ on a relevant neighborhood. Stability compares explanations for nearby inputs, e.g. $\|E(x) - E(x + \delta)\| / \|\delta\|$.

7. Practical Implementation

```
import numpy as np
from sklearn.inspection import permutation_importance
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import load_breast_cancer

X, y = load_breast_cancer(return_X_y=True, as_frame=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, stratify=y, test_size=0.25, random_state=42
```

```

)
model = RandomForestClassifier(n_estimators=200, random_state=42).fit(X_train,
y_train)

# Global, model-agnostic explanation on unseen data.
result = permutation_importance(
    model, X_test, y_test, scoring="roc_auc", n_repeats=10, random_state=42
)
ranking = X.columns[np.argsort(result.importances_mean)[::-1]][:8]
for name in ranking:
    j = X.columns.get_loc(name)
    print(name, round(result.importances_mean[j], 4), "+/-",
round(result.importances_std[j], 4))

```

8. Code Explanation

Permutation importance measures the decrease in held-out ROC-AUC after shuffling one feature. A useful feature loses its relationship with the outcome and damages performance. Repeats estimate variability. It is global and model-agnostic, but correlated features can substitute for one another and make each appear less important; conditional permutation or grouped features may be better.

9. Training / Evaluation

Explanations do not replace predictive evaluation. Use unseen representative data and evaluate: fidelity to the model; robustness across seeds/nearby inputs; sparsity and comprehensibility; coverage; and task-based human performance. For saliency, run model/label randomization tests. For counterfactuals, assess validity, distance, sparsity, feasibility, diversity, and whether recommended changes are actionable.

10. Complexity and Cost

Permutation importance needs roughly $O(Rd)$ evaluation passes. Model-agnostic local methods need many model queries per instance. Gradient explanations need one or several forward/backward passes. Exact combinatorial attribution is exponential, motivating approximations. Human studies and domain validation often cost more than compute.

11. Common Use Cases

- Debugging spurious correlations and leakage
- Explaining adverse credit or hiring decisions
- Clinical decision support and scientific discovery
- Validating image models attend to plausible regions
- Explaining recommender rankings
- Inspecting LLM/RAG citations, retrieval influence, or token sensitivity

12. Common Mistakes

- Equating explanation plausibility with faithfulness
- Treating association-based attribution as causality
- Using training data for importance and overstating generalization

- Ignoring correlated/encoded features and baseline choice
- Showing raw technical values to affected users without recourse context
- Reporting only cherry-picked examples
- Using explanations to justify an invalid model
- Exposing sensitive features or training examples through explanations

13. Edge Cases / Limitations

Equivalent predictions can have different internal rationales; correlated features make attribution non-identifiable. Explanations can be unstable, manipulated, or too complex. LLM chain-of-thought text need not reveal the model's true computation. Counterfactual changes can be impossible or socially harmful. A faithful explanation of a biased model remains biased.

14. Variations

- **Permutation/PDP/ALE:** global tabular behavior; ALE handles correlation better than PDP.
- **SHAP/LIME:** local post-hoc attribution; placement-essential.
- **Integrated Gradients/Grad-CAM:** neural NLP/CV attribution; project-important.
- **Counterfactual/recourse methods:** actionable decision changes; high-stakes systems.
- **Concept activation vectors:** influence of human concepts; research-oriented.
- **Mechanistic interpretability:** circuits and internal representations; advanced LLM research.

15. Related Topics

Interpretability is a property of understandable model structure; explainability often uses a post-hoc method. SHAP uses cooperative-game attribution, while LIME fits a local surrogate. Fairness uses explanations to locate proxies but requires outcome metrics too. Model cards communicate evaluated behavior; causal inference is needed before turning attribution into intervention advice.

16. Interview Questions

1. **Explainability versus interpretability?** Explainability communicates reasons, often post hoc; interpretability is direct human understanding of model mechanics.
2. **Local versus global?** Local explains one neighborhood/prediction; global summarizes behavior across a distribution.
3. **What is fidelity?** How accurately the explanation represents the actual model.
4. **Why can feature importance mislead?** Correlation, leakage, baselines, interactions, and distribution shift change attribution.
5. **Does SHAP prove causality?** No; standard SHAP explains associations under a background distribution.
6. **What makes a counterfactual useful?** Validity, minimality, feasibility, actionability, stability, and diversity.
7. **How test saliency?** Parameter/label randomization, deletion/insertion, localization where appropriate, and stability.
8. **Why explain on test data?** Training explanations may describe memorization and overstate deployed behavior.
9. **Can explanations create risk?** They can leak sensitive data, enable gaming, or create false trust.

10. **How choose a method?** Begin with question, audience, modality, model access, risk, and a measurable fidelity criterion.

17. Practice Tasks

- Compare permutation importance, coefficients, and SHAP on correlated tabular features.
- Build a local counterfactual generator with immutable-feature constraints.
- Run a saliency randomization sanity check on a CNN.
- Debug a model that relies on a post-outcome leakage feature.
- Conduct a small user study comparing two explanation formats.

18. Project Ideas

Project	What it does	Stack and dataset	Resume value
Explanation Reliability Lab	Compares fidelity/stability across methods	sklearn, SHAP, LIME; Adult	Rigorous XAI evaluation
Visual Diagnosis Inspector	Produces Grad-CAM plus deletion tests	PyTorch; Chest X-ray or skin lesion set	CV safety and XAI
Credit Recourse Tool	Generates constrained actionable counterfactuals	DiCE/sklearn, FastAPI; German Credit	User-centered responsible AI

19. Quick Revision

- **Key idea:** answer a defined question about model behavior for a defined audience.
- **Formula:** additive attribution $g = \phi_0 + \sum_j \phi_j z'_j$.
- **Use:** debugging, recourse, audit, trust calibration.
- **Metrics:** fidelity, stability, sparsity, validity, human task performance.
- **Trap:** plausible is not necessarily faithful or causal.
- **One-liner:** Evaluate the explanation as carefully as the prediction.

20. Final Cheat Sheet

Item	Summary
Definition	Communication of model reasoning or behavior
Input/output	Model, instance/distribution, background → attribution/rule/example/counterfactual
Steps	Define question → choose method → generate → validate → communicate limits
Hyperparameters	Baseline, neighborhood, samples, distance, sparsity
Metrics	Fidelity, stability, comprehensibility, recourse validity
Pros/cons	Helps debug and communicate; can be unstable, noncausal, or manipulable
Best use	High-impact predictions, debugging, scientific and compliance workflows

Interpretability

1. Overview

Interpretability is the degree to which a person can directly understand a model's inputs, internal relationships, and outputs. Linear models, small decision trees, rule lists, monotonic models, and generalized additive models can be intrinsically interpretable. Interpretability is valuable when humans must verify logic, contest decisions, discover relationships, or operate under strict safety constraints.

2. Intuition

An interpretable model is a transparent recipe: you can inspect each ingredient and step. A post-hoc explanation of a black box is more like asking a food critic to infer the recipe from tastes—useful, but approximate.

3. Prerequisites

- Linear/logistic regression and decision trees
- Feature engineering, scaling, and encoding
- Regularization and generalized additive models
- Domain semantics and human cognitive constraints
- Explainability and calibration basics

4. Core Concepts

Concept	Meaning and importance	Example	Interview angle
Simulatability	A human can execute the whole model	Depth-3 tree	Model size determines feasibility
Decomposability	Inputs, parameters, and components have meanings	Sparse logistic coefficients	Encoded features can destroy semantics
Algorithmic transparency	Training behavior is understood	Convex logistic regression	Different from understanding every prediction
Sparsity	Few active terms/rules	Five nonzero coefficients	Improves inspection, may reduce accuracy
Monotonicity	Prediction moves only in an allowed direction	Risk cannot fall as debt increases	Encodes domain constraints
Additivity	Output is sum of feature effects	GAM risk score	Easy plots, limited interactions
Interpretability–accuracy trade-off	Complexity can improve fit but reduce inspection	Tree versus boosted trees	Trade-off is empirical, not universal

5. Algorithm / Working Process

1. Define who must understand what and within what time.
2. Create semantically meaningful, non-leaky features.
3. Select the simplest model family able to meet performance and constraint needs.
4. Fit with sparsity, depth, monotonicity, or interaction limits.
5. Inspect coefficients, rules, shapes, and representative decisions with domain experts.
6. Compare against a strong black-box baseline to quantify any utility cost.
7. Monitor whether features and learned relationships retain their meanings after drift.

6. Mathematical Foundation

For logistic regression,

$$P(Y = 1 \mid x) = \sigma(\beta_0 + \beta^T x), \quad \frac{\text{odds}(x_j + 1)}{\text{odds}(x_j)} = e^{\beta_j}.$$

L1 regularization induces sparsity:

$$\min_{\beta} - \sum_i [y_i \log p_i + (1 - y_i) \log(1 - p_i)] + \lambda \|\beta\|_1.$$

A generalized additive model uses

$$g(\mathbb{E}[Y \mid x]) = \beta_0 + \sum_j f_j(x_j),$$

so each learned feature shape can be plotted. Interpretability depends on representation: a coefficient is not meaningful if the feature is an opaque embedding dimension.

7. Practical Implementation

```

import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

numeric = ["income", "debt_ratio"]
categorical = ["employment_type"]
prep = ColumnTransformer([
    ("num", StandardScaler(), numeric),
    ("cat", OneHotEncoder(handle_unknown="ignore"), categorical),
])
model = make_pipeline(
    prep,
    LogisticRegression(penalty="l1", solver="liblinear", C=0.2)
)
# model.fit(train[numeric + categorical], train["default"])
# names = model[0].get_feature_names_out()
# print(pd.Series(model[-1].coef_[0], index=names).sort_values())

```

8. Code Explanation

The pipeline makes preprocessing part of the fitted model, preventing train/test inconsistency. Standardization makes numeric coefficient magnitudes more comparable. One-hot encoding retains category meaning. L1 regularization sets weak terms to zero; smaller C means stronger regularization. Coefficients are conditional associations, not causal effects, and one-hot reference/category semantics must be documented.

9. Training / Evaluation

Use nested or held-out validation to tune sparsity/depth without touching the test set. Report predictive metrics, calibration, number of nonzero terms/rules, maximum tree depth, monotonic-constraint violations, and expert comprehension. Compare a transparent candidate against a competitive black-box baseline; sometimes careful features make the transparent model equally accurate.

10. Complexity and Cost

Linear inference is $O(d)$, sparse linear inference $O(k)$, and a balanced tree $O(\text{depth})$. Training convex linear models is CPU-friendly; GAMs and optimal rule lists can be costlier. Human review time grows with terms, interactions, and feature complexity.

11. Common Use Cases

- Credit scorecards and underwriting rules
- Medical risk scores and clinical decision support
- Fraud triage rules
- Industrial safety controllers
- Scientific relationship discovery
- Baselines for high-stakes black-box models

12. Common Mistakes

- Calling a 5,000-node tree interpretable because it is technically inspectable
- Reading coefficient magnitude without scaling or encoding context
- Treating coefficients as causal
- Using leaky or meaningless engineered features
- Ignoring interactions and nonlinearities when a linear model underfits
- Sacrificing large utility without measuring whether users understand the simpler model
- Presenting probabilities without calibration

13. Edge Cases / Limitations

High-dimensional images, audio, text embeddings, and complex interactions resist compact direct interpretation. Sparse models can be unstable under collinearity. A simple model can still be unfair or wrong. Domain users may misunderstand log-odds, and transparency can enable strategic gaming or expose security-sensitive logic.

14. Variations

- **Sparse linear/scorecard models:** strongest placement baseline for tabular decisions.
- **Small decision trees/rule lists:** direct if shallow; good for operational policies.
- **GAM/GA2M:** nonlinear feature shapes with limited interactions; excellent project choice.
- **Monotonic boosted models:** more accurate while preserving directional constraints.
- **Prototype/case-based models:** reason through representative examples; useful for images and medicine.
- **Mechanistic interpretability:** reverse-engineers neural circuits; research-focused.

15. Related Topics

Explainability generates an account of a model; intrinsic interpretability makes its mechanism directly inspectable. Regularization provides sparsity. Feature engineering controls semantic meaning. Causal inference is required for intervention claims. Model cards state intended interpretability and limitations, while formal verification can prove properties of constrained transparent or neural systems.

16. Interview Questions

1. **What is interpretability?** The degree to which a human can directly understand model operation and decisions.
2. **Intrinsic versus post-hoc?** Intrinsic models are transparent by design; post-hoc tools approximate or summarize a model.
3. **Is linear regression always interpretable?** No; thousands of correlated or opaque features defeat human understanding.
4. **How does L1 help?** It encourages zero coefficients, reducing the number of active terms.
5. **How interpret a logistic coefficient?** A one-unit increase multiplies odds by e^{β_j} , holding other modeled features fixed.
6. **What is a GAM?** A link-transformed prediction expressed as a sum of learned one-feature functions.
7. **Does interpretability require lower accuracy?** Not necessarily; the trade-off depends on data, features, and requirements.

8. **How measure interpretability?** Structural proxies plus task-based human speed, accuracy, and agreement.
9. **Why use monotonic constraints?** They encode trusted domain directionality and prevent implausible local behavior.
10. **Can a transparent model be unsafe?** Yes; transparent logic can be biased, miscalibrated, brittle, or gamed.

17. Practice Tasks

- Fit L1 logistic models across α values and plot performance versus nonzero features.
- Compare a depth-limited tree, GAM, and boosted tree on a tabular dataset.
- Convert log-odds coefficients into user-readable odds ratios.
- Debug a sign reversal caused by multicollinearity.
- Add and verify monotonic constraints to a risk model.

18. Project Ideas

Project	What it does	Stack and dataset	Resume value
Transparent Credit Scorecard	Produces calibrated points and reason codes	sklearn; Give Me Some Credit	Deployable high-stakes ML
Explainable Readmission GAM	Shows nonlinear clinical risk curves	InterpretML; UCI diabetes	Health + interpretable modeling
Rule-Based Fraud Triage	Learns compact rules and abstains on ambiguity	sklearn/rulefit; IEEE-CIS subset	Operations and human-in-loop design

19. Quick Revision

- **Key idea:** model mechanics are directly understandable to the relevant user.
- **Formula:** logistic odds ratio e^{β_j} ; GAM $g(E[Y]) = \beta_0 + \sum f_j(x_j)$.
- **Use:** high-stakes decisions, audit, science, human-in-loop systems.
- **Metrics:** predictive quality, calibration, sparsity/depth, human comprehension.
- **Trap:** technically visible does not mean cognitively interpretable.
- **One-liner:** Interpretability is a property of the model–representation–audience combination.

20. Final Cheat Sheet

Item	Summary
Definition	Direct human understanding of model structure and behavior
Input/output	Semantic features → inspectable score, rule, tree path, or shape sum
Steps	Define audience → choose constraints → train → inspect → compare → monitor
Hyperparameters	L1 strength, tree depth, knots/bins, interactions, monotonic constraints
Metrics	Utility, calibration, sparsity, rule count, comprehension
Pros/cons	Auditable and efficient; may underfit and can still be biased

Item	Summary
Best use	Regulated tabular decisions, science, clinical/operational support

Privacy

1. Overview

Privacy in ML means limiting inappropriate collection, inference, exposure, and use of information about people throughout the model lifecycle. It covers raw data, labels, embeddings, checkpoints, prompts, logs, explanations, and outputs. Privacy matters because models can memorize records, APIs can reveal membership or attributes, and operational telemetry can expose more data than training itself.

2. Intuition

Privacy is not “hide the spreadsheet.” It is controlled information flow: collect only what is needed, use it for an agreed purpose, restrict who can see it, retain it only as long as needed, and make releases reveal as little about any one person as possible.

3. Prerequisites

- Threat modeling and access control
- Hashing versus encryption and key management
- ML training/inference pipelines
- Membership and model-inversion attacks
- Differential privacy basics and data-protection terminology

4. Core Concepts

Concept	Meaning / why it matters	Example	Interview angle
Data minimization	Collect only necessary fields	Do not store DOB when age band suffices	Reduces breach and compliance scope
Purpose limitation	Use data only for stated compatible purposes	Support chats not silently reused for ads	Governance, not just security
Pseudonymization	Replace direct identifiers with controlled tokens	Customer ID mapped in a secure vault	Re-identification remains possible
Anonymization	Irreversibly prevent reasonable re-identification	Hard for rich/high-dimensional data	“De-identified” is not automatically anonymous
Memorization	Model reproduces rare training content	LLM emits an email address	Test canaries and extraction attacks
Membership inference	Determine whether a record was trained on	Higher confidence for a known patient	Overfitting increases risk

Concept	Meaning / why it matters	Example	Interview angle
Model inversion	Infer sensitive attributes or representative inputs	Reconstruct facial traits	Restrict output detail and access
Privacy by design	Build controls into architecture and lifecycle	Local processing, retention limits, DP	Earlier controls are cheaper and stronger

5. Algorithm / Working Process

1. Inventory data and classify sensitivity, ownership, purpose, residency, and retention.
2. Map flows from collection through features, training, artifacts, serving, logs, and deletion.
3. Define attackers, access, assets, and harms.
4. Minimize fields and retention; obtain valid authorization/consent where required.
5. Apply encryption, least privilege, isolation, audit logs, secret management, and output controls.
6. Use privacy-enhancing techniques such as DP, aggregation, secure computation, or federated learning when justified.
7. Test extraction/membership risk and rehearse deletion, incident response, and model rollback.

6. Mathematical Foundation

Privacy risk is often framed as an attacker's distinguishing advantage. For a membership attack A ,

$$\text{Adv} = P(A = 1 \mid z \in D) - P(A = 1 \mid z \notin D).$$

Near-zero advantage is desirable. k -anonymity requires each quasi-identifier pattern to appear at least k times, but does not prevent attribute disclosure when each group shares the same secret. Differential privacy gives a formal neighboring-dataset bound:

$$P[M(D) \in S] \leq e^\epsilon P[M(D') \in S] + \delta.$$

Encryption protects data at rest/in transit; it does not constrain what an authorized training process learns or emits.

7. Practical Implementation

```
import hashlib, hmac, os

def pseudonymize(identifier: str, secret: bytes) -> str:
    # Keyed HMAC prevents cheap dictionary reversal unlike plain hashing.
    return hmac.new(secret, identifier.encode(), hashlib.sha256).hexdigest()

def privacy_safe_record(row: dict, secret: bytes) -> dict:
    return {
        "subject_token": pseudonymize(row["email"].lower().strip(), secret),
        "age_band": f"{{(row['age'] // 10) * 10}}s",
        "event": row["event"],
    }

secret = os.urandom(32) # In production, retrieve a versioned key from a KMS.
```

```
print(privacy_safe_record({"email": "a@example.com", "age": 27, "event": "click"},  
secret))
```

8. Code Explanation

HMAC creates a stable pseudonym only for holders of the secret, avoiding the dictionary weakness of plain hashes over emails. Age is coarsened and unnecessary fields are dropped. The output remains personal data because linkage and re-identification may still be possible. Production keys belong in a KMS, with rotation, access logging, separation of duties, and a deletion mapping.

9. Training / Evaluation

Measure model utility alongside privacy. Use deduplication, regularization, early stopping, DP accounting, canary exposure, membership-inference AUC/advantage, secret/PII scanners, and extraction-rate tests. Keep privacy test data separated from training. Evaluate logs and RAG indexes too. Red-team high-risk interfaces under realistic query budgets and privileges.

10. Complexity and Cost

Encryption/HMAC is linear in bytes and negligible compared with training. DP training adds clipping/noise and often more steps; secure computation and homomorphic encryption can be orders of magnitude slower. Data mapping, access reviews, and deletion propagation are substantial operational costs.

11. Common Use Cases

- Healthcare, finance, education, and HR models
- LLM training and chat telemetry
- Personalized recommendation and advertising
- Biometrics and location analytics
- Cross-organization research and federated analytics

12. Common Mistakes

- Calling pseudonymized data anonymous
- Hashing predictable identifiers without a secret
- Logging prompts, headers, retrieved documents, or errors indefinitely
- Copying production PII into development
- Believing encryption prevents model memorization
- Publishing embeddings as if they cannot leak source text
- Forgetting backups, caches, checkpoints, and downstream deletion
- Measuring compliance paperwork but not extraction risk

13. Edge Cases / Limitations

Rare records and high-dimensional data are readily linkable. Deletion from a trained model may require retraining or verified unlearning. Privacy and debugging/auditability can conflict. Consent may not make harmful secondary uses acceptable. Regulations vary by jurisdiction; obtain qualified legal guidance for actual compliance decisions.

14. Variations

- **Differential privacy:** formal release/training guarantee; placement and research important.
- **Federated learning:** keeps raw records local but does not itself guarantee privacy.
- **Secure aggregation/MPC:** hides individual updates; production/research relevant.
- **Trusted execution environments:** hardware-isolated processing; infrastructure option.
- **Synthetic data:** can reduce exposure but may memorize or distort minorities.
- **Machine unlearning:** removes influence of deletion requests; emerging research.

15. Related Topics

PII handling implements operational privacy controls. Differential privacy bounds individual influence; federated learning changes data location. Security protects confidentiality against unauthorized access, while privacy also limits authorized use. Dataset/model cards record consent, intended use, retention, and known leakage risks.

16. Interview Questions

1. **Privacy versus security?** Security prevents unauthorized access; privacy governs appropriate collection, use, disclosure, and inference.
2. **Pseudonymization versus anonymization?** Pseudonyms remain linkable personal data; robust anonymization prevents reasonable re-identification.
3. **Why is plain hashing emails weak?** The small predictable domain enables dictionary attacks.
4. **What is membership inference?** Determining whether a specific record influenced training.
5. **What increases membership risk?** Overfitting, rare records, excessive output detail, and unrestricted queries.
6. **Do embeddings preserve privacy?** No; inversion, linkage, and nearest-neighbor attacks may reveal sources.
7. **How handle deletion?** Delete sources/derivatives and retrain, unlearn, or document a justified lifecycle process.
8. **Does federated learning guarantee privacy?** No; gradients and outputs can leak without DP/secure aggregation.
9. **How test an LLM?** Canary exposure, PII probes/scanners, extraction attacks, memorization and access-control tests.
10. **Best privacy control?** Avoid collecting or retaining unnecessary data.

17. Practice Tasks

- Create a data inventory with purpose, sensitivity, owner, and retention.
- Implement HMAC tokenization and key-rotation versioning.
- Train an overfit classifier and measure membership attack AUC.
- Find PII leakage in application logs and propose minimum logging.
- Compare non-private and DP training utility/privacy curves.

18. Project Ideas

Project	What it does	Stack/dataset	Resume value
Privacy Gateway	Redacts/tokenizes PII before LLM calls	Presidio, FastAPI; synthetic support chats	AI engineering controls
Membership Audit Lab	Attacks and hardens image classifiers	PyTorch, Opacus; CIFAR-10	ML privacy research
Deletion Lineage Tracker	Traces records into features/artifacts	Python, MLflow; synthetic	MLOps governance

19. Quick Revision

- **Key idea:** minimize and control information flow across the entire lifecycle.
- **Formula:** attack advantage or the DP (ϵ, δ) bound.
- **Use:** any system processing personal or confidential data.
- **Metrics:** membership AUC/advantage, extraction rate, DP budget, deletion SLA.
- **Trap:** de-identification and encryption are not complete privacy guarantees.
- **One-liner:** The cheapest private datum is the one never collected.

20. Final Cheat Sheet

Item	Summary
Definition	Appropriate, limited information collection, use, inference, and disclosure
Input/output	Data flow + threat model → minimized, controlled pipeline
Steps	Inventory → minimize → protect → test → retain/delete → monitor
Hyperparameters	Retention, access scope, output precision, ϵ, δ
Metrics	Attack advantage, extraction, audit coverage, deletion latency
Pros/cons	Reduces harm/compliance risk; may reduce utility and observability
Best use	Personal, confidential, biometric, behavioral, and proprietary data

Data Leakage

1. Overview

Data leakage occurs when model training, feature construction, or evaluation uses information unavailable at the real prediction time or improperly shares information across splits. It produces deceptively strong validation results and failed deployments. Leakage is among the most frequent ML interview and production errors because it can hide inside timestamps, preprocessing, duplicate entities, target-derived features, and cross-validation code.

2. Intuition

Predicting an exam score using the answer key is easy but meaningless. Leakage is any hidden answer key—even a subtle one, such as a feature calculated after the decision or the same patient appearing in train and test.

3. Prerequisites

- Train/validation/test roles
- Feature pipelines and cross-validation
- Time series, grouped data, and deployment timestamps
- Target encoding and preprocessing
- Entity resolution and duplicate detection

4. Core Concepts

Type	Meaning / why	Example	Interview angle
Target leakage	Feature contains outcome or post-outcome signal	"Claim paid" predicts fraud	Ask when feature becomes known
Train-test contamination	Test information affects fitting	Scaling on all rows	Fit transformations inside folds
Entity leakage	Related records cross splits	Same patient visits in train/test	Group split by entity
Temporal leakage	Future informs past	Random split of churn events	Time-aware split and cutoff
Duplicate leakage	Exact/near duplicates cross splits	Augmented image variants	Deduplicate before splitting
Preprocessing leakage	Imputation/selection sees validation	Select features using full labels	Pipeline all learned steps
LLM benchmark contamination	Benchmark items appear in pretraining/fine-tuning	Memorized test answers	Decontamination and fresh tests
RAG leakage	Unauthorized/answer documents enter retrieval context	Gold answer indexed during evaluation	ACL and corpus-version audit

5. Algorithm / Working Process

1. Write the prediction timestamp and list exactly what is available then.
2. Define the independent unit (user, patient, device, document, location).
3. Deduplicate and split raw data by time/group before learned preprocessing.
4. Put imputation, scaling, encoding, selection, and resampling inside a pipeline fitted per fold.
5. Audit suspicious features for post-event timestamps, target correlation, names, IDs, and provenance.
6. Compare random versus realistic splits; a dramatic collapse is a warning.
7. Freeze the test set and log dataset/query/corpus versions.

6. Mathematical Foundation

The valid generalization estimate is

$$\hat{R}_{test} = \frac{1}{m} \sum_{(x,y) \in D_{test}} \ell(f_{D_{train}}(x), y),$$

where every fitted parameter of f , including preprocessing, depends only on training data. If $T(D_{train} \cup D_{test})$ computes a scaler or feature selector, independence is broken and $\hat{R}_{test}$ is optimistically biased. For time t_i , every feature must obey availability time $\tau(x_{ij}) \leq t_i$.

7. Practical Implementation

```
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import GroupKFold, cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

numeric, categorical = ["age", "balance"], ["region"]
prep = ColumnTransformer([
    ("num", make_pipeline(SimpleImputer(), StandardScaler()), numeric),
    ("cat", OneHotEncoder(handle_unknown="ignore"), categorical),
])
pipeline = make_pipeline(pre, LogisticRegression(max_iter=1000))

# All visits from one customer remain in one fold; preprocessing is refit per
# fold.
cv = GroupKFold(n_splits=5)
# scores = cross_val_score(pipeline, X, y, groups=customer_id, cv=cv,
# scoring="roc_auc")
```

8. Code Explanation

Pipeline prevents the imputer, scaler, and encoder from learning validation statistics. **GroupKFold** prevents repeated customer records from crossing fold boundaries. This does not solve temporal leakage automatically: if deployment predicts future customers/events, add an explicit chronological cutoff or rolling-origin evaluation.

9. Training / Evaluation

Make splits reflect inference: group split for repeated entities, forward-chaining for time, site holdout for geographic/generalization claims. Nested CV is needed when reporting performance after extensive tuning. Keep resampling such as SMOTE inside each training fold. Run a label-shuffle test—performance should fall to chance—and compare against suspicious single-feature baselines.

10. Complexity and Cost

Correct cross-validation costs roughly k training runs. Group/time splits may reduce effective sample size and increase variance, but that is honest uncertainty. Duplicate detection ranges from $O(n)$ hashing for exact matches to approximate-neighbor cost for semantic duplicates.

11. Common Use Cases

- Patient-level medical prediction
- Churn, fraud, credit, and predictive maintenance over time
- Image classification with augmented/near-duplicate media
- Target encoding and feature selection
- LLM benchmark and RAG evaluation
- Competition models whose leaderboard does not match deployment

12. Common Mistakes

- Scaling, imputing, or selecting features before splitting
- Applying SMOTE before cross-validation
- Randomly splitting time-dependent data
- Ignoring multiple rows per person/device
- Tuning on the test set through repeated reporting
- Using post-outcome billing/status fields
- Confusing a feature's event timestamp with its actual availability timestamp
- Deduplicating only exact text when templated near-duplicates exist

13. Edge Cases / Limitations

Some tasks intentionally use transductive information; state that assumption. Global unsupervised pretraining can still contaminate a benchmark. Temporal embargoes may be necessary when labels overlap future windows. Entity identity can be hidden or probabilistic. Leakage tests are diagnostic, not proofs of absence.

14. Variations

- **Group-aware splitting:** repeated entities; placement-essential.
- **TimeSeriesSplit/rolling origin:** forecasting and event prediction.
- **Purged/embargo CV:** finance with overlapping label windows; advanced.
- **Nested CV:** unbiased model-selection assessment; interview-important.
- **Benchmark decontamination:** n-gram/minhash/semantic matching for LLMs.
- **ACL-aware RAG evaluation:** tests both answer quality and authorization boundaries.

15. Related Topics

Data preprocessing must be fold-local. Distribution shift is real change after deployment, whereas leakage is invalid information during development. Evaluation benchmarks require contamination controls. MLOps lineage records exactly which snapshot and transformation produced an artifact. Privacy leakage concerns confidential disclosure; data leakage here primarily means evaluation/training contamination, though they can overlap.

16. Interview Questions

1. **What is data leakage?** Use of unavailable or improperly shared information that inflates estimated performance.
2. **Target versus train-test leakage?** Target leakage uses outcome-derived signals; contamination lets held-out data influence fitting.
3. **Why scale inside a pipeline?** Each fold's scaler must learn only its training rows.
4. **Why random split can fail?** Time, entity, site, or duplicate dependence violates IID assumptions.
5. **Where should SMOTE run?** Only on each training fold after splitting.
6. **How detect leakage?** Provenance/timestamp audits, split comparisons, single-feature tests, label shuffle, duplicate search.
7. **Can unsupervised preprocessing leak?** Yes; it estimates held-out distribution and can bias model selection.
8. **What is test-set overfitting?** Repeated decisions based on test results turn the test set into validation data.
9. **How handle target encoding?** Learn encodings out-of-fold for training and from training only for validation/test.
10. **LLM contamination mitigation?** Search overlaps, use held-out/private/fresh tasks, track snapshots, and test robustness beyond exact recall.

17. Practice Tasks

- Demonstrate inflated accuracy from scaling/selection before CV, then fix it.
- Build customer-group and temporal splits for churn data.
- Detect exact and near-duplicate text across splits.
- Debug a medical model using discharge-only fields.
- Create an out-of-fold target encoder.

18. Project Ideas

Project	What it does	Stack/dataset	Resume value
Leakage Linter	Flags post-outcome names, duplicates, split overlap, and preprocessing	Pandas, sklearn	Practical ML quality tooling
Temporal Churn Evaluator	Compares random and rolling evaluation	LightGBM/sklearn; Telco synthetic events	Production evaluation design
LLM Decontaminator	Finds n-gram and MinHash benchmark overlap	datasketch, HF datasets	LLM evaluation engineering

19. Quick Revision

- **Key idea:** evaluation must use only information available at real inference time.
- **Formula:** test loss of a model fit exclusively on training data.
- **Use:** every ML pipeline.
- **Metrics:** realistic holdout performance, overlap rate, shuffled-label score.
- **Trap:** split raw data before every learned transform.
- **One-liner:** If the model or pipeline saw the answer key, the metric is fiction.

20. Final Cheat Sheet

Item	Summary
Definition	Invalid future, target, entity, duplicate, or holdout information flow
Input/output	Raw timestamped/grouped data → isolated realistic splits and pipeline
Steps	Define inference → group/time split → pipeline transforms → audit → freeze test
Hyperparameters	Cutoff, gap/embargo, folds, group key, duplicate threshold
Metrics	Holdout score, overlap, feature availability violations
Pros/cons	Honest estimates can be lower/noisier but prevent deployment surprise
Best use	Universal; crucial for temporal, repeated-entity, LLM benchmark tasks

Model Robustness

1. Overview

Model robustness is the ability to maintain acceptable behavior under realistic perturbations, corruption, distribution shift, missing inputs, hardware noise, and deliberate attack. Robustness is not one scalar property: a model can resist Gaussian noise but fail on a new hospital, or withstand common image corruptions but not adversarial examples.

2. Intuition

A bridge is not judged only on a calm day; engineers specify loads, wind, temperature, and safety margins. Likewise, ML robustness begins with an operating envelope and explicit failure conditions.

3. Prerequisites

- Generalization, regularization, and calibration
- Data augmentation and distribution shift
- Norms, gradients, and optimization
- Stress testing and uncertainty
- Monitoring and fallback design

4. Core Concepts

Concept	Meaning / why	Example	Interview angle
Natural corruption	Non-malicious degradation	Blur, noise, typo	Evaluate severity curves
Covariate shift	$p(x)$ changes	New camera/site	Reweight only under assumptions
Label/concept shift	$p(y)$ or $p(y x)$ changes		Fraud tactics change

Concept	Meaning / why	Example	Interview angle
Worst-group robustness	Protect weakest known slice	Rare weather condition	Average can hide failure
OOD detection	Identify unsupported inputs	Novel disease/image	Scores are imperfect
Calibration/selective prediction	Confidence reflects risk; abstain when uncertain	Human review below confidence	Coverage–risk trade-off
Certified robustness	Mathematical guarantee inside perturbation set	Radius around an image	Narrow threat-model guarantee
Operational resilience	Safe fallback, rollback, monitoring	Disable automation on drift	Model robustness alone is insufficient

5. Algorithm / Working Process

1. Define intended distribution, perturbations, severities, shifts, and unacceptable failures.
2. Establish clean performance and slice baselines.
3. Build stress sets from natural corruptions, time/site holdouts, missing fields, and attacks.
4. Plot metric versus severity and measure worst-group and calibration degradation.
5. Improve data coverage, augmentation, objective, architecture, uncertainty, or fallback.
6. Re-test against adaptive and unseen perturbations.
7. Deploy monitors, abstention/routing, rollback, and incident ownership.

6. Mathematical Foundation

Standard empirical risk minimizes average loss; robust optimization considers a perturbation set Δ :

$$\min_{\theta} \mathbb{E}_{(x,y)} \left[\max_{\delta \in \Delta} \ell(f_{\theta}(x + \delta), y) \right].$$

Distributionally robust optimization uses

$$\min_{\theta} \sup_{Q: d(Q,P) \leq \rho} \mathbb{E}_Q[\ell(f_{\theta}(X), Y)].$$

Robust accuracy is accuracy under a specified attack/corruption. Relative degradation can be $(M_{clean} - M_{stress})/M_{clean}$. Selective risk at coverage c evaluates error only where confidence exceeds a threshold; lower coverage can yield safer decisions.

7. Practical Implementation

```
import numpy as np
from sklearn.metrics import accuracy_score

def corruption_curve(model, X, y, sigmas=(0.0, 0.05, 0.1, 0.2), seed=42):
    rng = np.random.default_rng(seed)
    rows = []
```

```

    scale = np.asarray(X).std(axis=0) + 1e-12
    for sigma in sigmas:
        noisy = np.asarray(X) + rng.normal(size=np.asarray(X).shape) * scale *
sigma
        rows.append((sigma, accuracy_score(y, model.predict(noisy))))
    return rows

# Report the full curve, not only one hand-picked severity.
# print(corruption_curve(model, X_test, y_test))

```

8. Code Explanation

Noise is scaled per feature so one common severity has comparable meaning across units. A fixed random generator makes tests reproducible. The curve reveals graceful or sudden degradation. Gaussian noise is only one stressor; use domain-valid corruptions and never claim adversarial robustness from this test.

9. Training / Evaluation

Keep clean and stress sets separate. Evaluate performance, calibration, worst slice, failure severity, and risk-coverage curves. Use multiple corruption types and unseen severities. Augmentation must occur only in training. Compare in-distribution, temporal/site shift, and OOD. For adversarial claims, use strong adaptive attacks and report threat model, norm, budget, steps, and restarts.

10. Complexity and Cost

Testing costs $O(Kn)$ inference for K stresses. Adversarial training may multiply training cost by attack steps. Ensembles improve uncertainty but multiply inference/memory. Simple corruption suites are CPU/GPU-parallel; certification can be much more expensive.

11. Common Use Cases

- Autonomous driving across weather/sensors
- Medical models across hospitals and devices
- ASR/NLP under accents, typos, paraphrases, and noise
- Fraud/spam under evolving tactics
- Industrial predictive maintenance with missing sensors
- LLM/RAG reliability under prompt and retrieval variation

12. Common Mistakes

- Reporting only clean IID accuracy
- Using unrealistic perturbations and claiming general robustness
- Tuning on the stress test until it becomes training data
- Averaging away catastrophic slices
- Confusing confidence with correctness
- Treating OOD detector scores as guarantees
- Evaluating a weak attack that causes gradient masking
- Improving robustness while ignoring clean utility/calibration cost

13. Edge Cases / Limitations

The space of shifts is unbounded; passing a suite guarantees only that suite. Robustness can trade off with accuracy, fairness, latency, or privacy. Some shifts change the label itself, so invariance is wrong. OOD detection is difficult near the training boundary, and adaptive attackers target the complete deployed pipeline.

14. Variations

- **Augmentation/corruption training:** practical baseline for vision/audio/NLP.
- **Adversarial training:** strong empirical local robustness; costly.
- **DRO/worst-group training:** handles distribution or group uncertainty.
- **Randomized smoothing:** certified L_2 robustness; research/interview relevant.
- **Ensembles/uncertainty:** improve failure detection at extra cost.
- **Conformal prediction:** coverage guarantees under exchangeability; important emerging topic.

15. Related Topics

Adversarial examples are deliberately optimized robustness failures. Distribution shift and drift monitoring operationalize natural robustness. Fairness often uses worst-group robustness. Formal verification proves bounded properties. Red teaming finds failure modes beyond a fixed benchmark. Calibration and uncertainty enable abstention.

16. Interview Questions

1. **What is robustness?** Acceptable performance under a specified family of deviations from nominal conditions.
2. **Robustness versus generalization?** Generalization concerns unseen samples from an assumed distribution; robustness explicitly tests deviations/shifts.
3. **Covariate versus concept shift?** $p(x)$ changes versus $p(y|x)$ changes.
4. **What is robust accuracy?** Accuracy under a declared attack or corruption, not a universal score.
5. **How evaluate?** Define threat/shift model, stress severities, slices, calibration, worst-case results, and uncertainty.
6. **What is selective prediction?** Abstain or route cases when confidence/risk fails a criterion.
7. **Why can augmentation hurt?** It may create label-changing or unrealistic transformations.
8. **What is DRO?** Optimization for worst expected loss over a neighborhood of distributions.
9. **Empirical versus certified robustness?** Attack-tested evidence versus a mathematical guarantee within assumptions.
10. **What belongs in production?** Stress-tested model plus monitoring, fallback, rollback, and incident response.

17. Practice Tasks

- Build corruption curves for noise, missingness, blur, and feature scaling.
- Evaluate a classifier on geographic or temporal holdouts.
- Plot risk versus coverage for confidence-based abstention.
- Diagnose apparent robustness caused by a weak attack.
- Train with augmentation and test on unseen corruption types.

18. Project Ideas

Project	What it does	Stack/dataset	Resume value
Robustness Workbench	Generates stress suites and degradation reports	sklearn/PyTorch; CIFAR-10-C	Evaluation engineering
Safe Sensor Classifier	Handles missing/noisy sensors with abstention	PyTorch; HAR/UCI	Edge/safety ML
Cross-Site Medical Test	Measures site shift and worst-group calibration	sklearn; public clinical data	Realistic validation design

19. Quick Revision

- **Key idea:** robustness is always relative to a declared perturbation/shift set.
- **Formula:** minimax loss $\min_{\theta} E[\max_{\delta \in \Delta} \ell]$.
- **Use:** variable or adversarial deployment environments.
- **Metrics:** robust accuracy, degradation, worst-group risk, calibration, coverage.
- **Trap:** one corruption suite does not imply universal robustness.
- **One-liner:** Specify the operating envelope before claiming a model is robust.

20. Final Cheat Sheet

Item	Summary
Definition	Maintains acceptable behavior under specified deviations
Input/output	Model + stress/threat suite → degradation and failures
Steps	Scope → baseline → stress → mitigate → retest → monitor/fallback
Hyperparameters	Severity, norm budget, DRO radius, abstention threshold
Metrics	Clean/robust score, worst slice, ECE, risk–coverage
Pros/cons	Improves reliability; testing is incomplete and mitigation costs utility/compute
Best use	Safety-critical, shifting, noisy, and adversarial settings

Adversarial Examples

1. Overview

Adversarial examples are inputs intentionally modified to cause a model error while respecting an attacker-defined constraint. They occur in vision, audio, malware, tabular systems, NLP, and embodied AI. Studying them reveals brittle decision boundaries and supports realistic security testing, but “imperceptible L_p noise” is only one threat model.

2. Intuition

An attacker finds the model's optical illusion: a small or plausible change that humans consider equivalent but that crosses the learned decision boundary. Security depends on attacker knowledge, capabilities, query budget, objective, and whether the perturbation survives the physical/deployed pipeline.

3. Prerequisites

- Gradients, loss functions, and backpropagation
- Vector norms and constrained optimization
- Classification logits and cross-entropy
- White-box versus black-box security models
- PyTorch inference and evaluation

4. Core Concepts

Concept	Meaning / why	Example	Interview angle
Evasion attack	Changes inference input	Perturbed stop sign	Most common adversarial example
Targeted/untargeted	Force chosen class / any wrong class	Make "cat" become "dog" / not-cat	Targeted is usually harder
White/black/gray box	Full, query-only, or partial model knowledge	Transfer from surrogate	State access assumptions
Perturbation constraint	Bounds allowed change	$ \delta _{\infty} \leq \epsilon$	Norm may not match real semantics
FGSM/PGD	One-step / iterative gradient attacks	Pixel perturbation	PGD is stronger first-order test
Transferability	Attack on one model fools another	Surrogate attack	Enables black-box attacks
Physical attack	Survives camera/environment transforms	Adversarial patch	Evaluate transformations
Poisoning/backdoor	Training-time manipulation	Trigger causes target class	Related but not inference-only example

5. Algorithm / Working Process

For PGD: start from x (optionally random within the budget), compute the gradient of loss with respect to input, move in the loss-increasing direction, project back into the allowed norm ball and valid input range, repeat, and retain successful/high-loss examples. Evaluate on correctly classified clean inputs with several restarts and adaptive access to the full defense.

6. Mathematical Foundation

An untargeted attack solves

$$\max_{\delta} \ell(f_{\theta}(x + \delta), y) \quad \text{s.t.} \quad \|\delta\|_p \leq \epsilon, x + \delta \in [0, 1]^d.$$

FGSM uses $x' = \text{clip}(x + \epsilon \text{sign}(\nabla_x \ell), 0, 1)$. PGD iterates

$$x^{t+1} = \Pi_{B_p(x, \epsilon)} (x^t + \alpha \text{sign}(\nabla_{x^t} \ell)) .$$

Targeted attacks minimize loss for target class. Adversarial training approximately minimizes the inner maximum over attacks.

7. Practical Implementation

```
import torch

def fgsm(model, x, y, epsilon):
    x_adv = x.detach().clone().requires_grad_(True)
    loss = torch.nn.functional.cross_entropy(model(x_adv), y)
    model.zero_grad(set_to_none=True)
    loss.backward()
    return (x_adv + epsilon * x_adv.grad.sign()).clamp(0, 1).detach()

def robust_accuracy(model, loader, epsilon=8/255):
    correct = total = 0
    model.eval()
    for x, y in loader:
        x_adv = fgsm(model, x, y, epsilon)
        with torch.no_grad():
            correct += (model(x_adv).argmax(1) == y).sum().item()
            total += y.numel()
    return correct / total
```

8. Code Explanation

The input, not model parameters, requires gradients. Cross-entropy is maximized by stepping with the gradient sign. Clamping enforces valid normalized pixel range. `detach` prevents graph accumulation. This is a minimal white-box FGSM test; serious claims require PGD/AutoAttack, correct normalization-aware bounds, multiple budgets, and evaluation of the entire deployed preprocessing pipeline.

9. Training / Evaluation

Report clean accuracy, robust accuracy, attack success rate, targeted success, perturbation budget/norm, steps, step size, restarts, and attacker knowledge. Attack only clean-correct cases when reporting success and state the denominator. Check gradient masking with stronger, gradient-free, transfer, and adaptive attacks. Keep an unseen attack configuration for final evaluation.

10. Complexity and Cost

FGSM costs roughly one backward pass per batch. A T -step PGD attack costs T backward passes; adversarial training has similar multiplicative overhead. Black-box attacks can require thousands of queries. Certified defenses are often substantially more expensive.

11. Common Use Cases

- Security evaluation of image classifiers and biometrics
- Malware/spam/fraud evasion
- Adversarial patches for physical perception systems
- Robust speech recognition
- Stress-testing content classifiers and safety filters
- Testing ML APIs against surrogate/transfer attacks

12. Common Mistakes

- Attacking probabilities after nondifferentiable preprocessing without adapting the attack
- Forgetting input normalization when setting ϵ
- Claiming defense from FGSM alone
- Including already-misclassified clean samples in attack success
- Causing gradient masking and mistaking attack failure for robustness
- Using perceptual similarity claims based only on L_p
- Evaluating only the base model, not preprocessing/ensemble/randomization

13. Edge Cases / Limitations

Small norm does not guarantee semantic equivalence; large semantic changes can have small task impact and vice versa. Discrete text/tabular constraints require valid edits. Physical attacks face printing, view, and lighting transformations. Defenses can overfit known attacks. A robustness certificate holds only for its norm, radius, and assumptions.

14. Variations

- **FGSM**: fast baseline; placements.
- **PGD/AutoAttack**: stronger empirical evaluation; projects/research.
- **CW attack**: optimization-based targeted attack.
- **Black-box transfer/query attacks**: realistic API threats.
- **Adversarial patches/EOT**: physical attacks across transformations.
- **Randomized smoothing/IBP**: certified robustness; advanced research.
- **Poisoning/backdoors**: training-time adversarial manipulation.

15. Related Topics

Robustness provides the broader reliability frame. Red teaming selects realistic attacker goals and system surfaces. Formal verification can certify bounded robustness. Prompt injection and jailbreaks are language-system attacks but cannot be reduced neatly to pixel-norm perturbations. Data poisoning targets training rather than inference.

16. Interview Questions

1. **What is an adversarial example?** An intentionally modified input designed to cause a model failure under specified constraints.
2. **FGSM formula?** $x' = x + \epsilon \text{sign}(\nabla_x \ell)$, clipped to valid range.
3. **Why sign gradient?** Under an L_∞ budget it maximizes the first-order loss increase.
4. **FGSM versus PGD?** One step versus repeated projected steps; PGD is generally stronger.

5. **White-box versus black-box?** Full gradient/model access versus query or transfer access.
6. **What is transferability?** Perturbations crafted on one model fool another.
7. **How identify gradient masking?** Iterative/black-box attacks outperform gradients, robustness behaves oddly with budget, or unbounded attack fails.
8. **What is adversarial training?** Training on inner-maximized adversarial examples to minimize worst-case loss.
9. **Empirical versus certified defense?** Tested resistance versus proved resistance inside a formal region.
10. **How report attack success?** With threat model, denominator, budget, norm, method, steps, restarts, and clean/robust scores.

17. Practice Tasks

- Implement FGSM and PGD in PyTorch and plot accuracy versus ϵ .
- Check attacks with/without correct normalization.
- Compare transfer attacks across two architectures.
- Diagnose a defensive preprocessing layer for gradient masking.
- Adversarially train a small CNN and measure clean–robust trade-off.

18. Project Ideas

Project	What it does	Stack/dataset	Resume value
Attack/Defense Arena	Reproducible FGSM, PGD, AutoAttack reports	PyTorch, ART; CIFAR-10	Security evaluation rigor
Physical Patch Study	Tests patches under geometric/lighting transforms	torchvision/OpenCV; traffic signs	CV security research
Tabular Evasion Lab	Enforces immutable/range/categorical constraints	sklearn, ART; fraud data	Realistic business threat modeling

19. Quick Revision

- **Key idea:** optimize an allowed input change to cross the decision boundary.
- **Formula:** constrained loss maximization; FGSM gradient-sign step.
- **Use:** security and robustness testing.
- **Metrics:** clean/robust accuracy, attack success, budget, queries.
- **Trap:** weak or incorrectly configured attacks create false security.
- **One-liner:** An adversarial result is meaningful only with its threat model.

20. Final Cheat Sheet

Item	Summary
Definition	Deliberately perturbed input causing failure
Input/output	Model, input, label/target, budget → adversarial input
Steps	Gradient/query → update → project → clip → repeat/evaluate

Item	Summary
Hyperparameters	ϵ , norm, step size, steps, restarts, query budget
Metrics	Robust accuracy, success rate, distortion, queries
Pros/cons	Reveals brittleness; constraints may be unrealistic and attacks incomplete
Best use	Security testing of differentiable and queryable ML systems

Hallucination

1. Overview

Hallucination is generated content that is unsupported by the input, trusted evidence, world state, or task constraints while being presented as valid. In LLMs it includes invented facts, citations, tool results, entities, and reasoning steps; in image generation it can mean anatomically or semantically inconsistent content. Hallucination matters in search, RAG, agents, medicine, law, coding, and customer support because fluent language can conceal uncertainty.

2. Intuition

An LLM is an excellent next-token predictor, not a built-in fact database. When evidence is absent or competing continuations are plausible, it can complete the pattern confidently—like an articulate student improvising an answer instead of saying “I do not know.”

3. Prerequisites

- Autoregressive language modeling and decoding
- Probability, entropy, and calibration
- Retrieval-augmented generation (RAG)
- Grounding, citations, tool calling, and structured output
- Human and automated evaluation

4. Core Concepts

Concept	Meaning / why	Example	Interview angle
Intrinsic hallucination	Contradicts supplied source	Summary reverses revenue trend	Test entailment against context
Extrinsic hallucination	Adds unsupported claim	Invents acquisition not in source	May be true globally but ungrounded
Factuality vs faithfulness	Correct in world vs supported by provided evidence	True fact absent from document	Define evaluation target
Citation hallucination	Citation nonexistent or does not support claim	Fabricated DOI	Verify existence and entailment separately

Concept	Meaning / why	Example	Interview angle
Retrieval failure	Relevant evidence not retrieved	Wrong top-k chunk	Separate retriever and generator errors
Generation failure	Evidence present but ignored/misused	Wrong dosage despite correct passage	Evaluate context utilization
Abstention	Declines when evidence insufficient	"Sources do not establish this"	Risk–coverage trade-off
Self-consistency	Repeated samples disagree	Different dates across runs	Signal, not truth proof

5. Algorithm / Working Process

1. Define what counts as truth: source documents, database/tool results, or adjudicated references.
2. Decompose system into retrieval, evidence selection, generation, citation, and presentation.
3. Require claims to carry source identifiers/spans or structured provenance.
4. Verify each atomic claim for entailment and each citation for existence/support.
5. Test answerable and unanswerable questions, conflicts, stale evidence, and prompt attacks.
6. Calibrate abstention/escalation thresholds using validation risk and business cost.
7. Monitor unsupported-claim rate with human review of high-risk outputs.

6. Mathematical Foundation

Autoregressive generation models

$$P(y \mid x) = \prod_{t=1}^T P(y_t \mid y_{<t}, x).$$

Maximum likelihood rewards likely text, not external truth. Claim precision is

$$\text{groundedness} = \frac{\# \text{entailed generated claims}}{\# \text{generated claims}},$$

while evidence coverage is supported reference claims divided by required reference claims. For selective answering, choose abstention threshold τ to minimize

$$R(\tau) = C_{\text{wrong}} P(\text{wrong}, \text{answer}) + C_{\text{abstain}} P(\text{abstain}).$$

Token probability alone is poorly calibrated for semantic factuality.

7. Practical Implementation

```
from dataclasses import dataclass

@dataclass
class Answer:
    text: str
    source_ids: list[str]
```

```

confidence: float

def guarded_answer(question, retrieved, generate, threshold=0.7):
    if not retrieved:
        return Answer("Insufficient evidence.", [], 0.0)
    draft = generate(question, retrieved) # Must return source IDs it actually
    used.
    valid_ids = {doc["id"] for doc in retrieved}
    citations_valid = bool(draft.source_ids) and set(draft.source_ids) <=
    valid_ids
    if draft.confidence < threshold or not citations_valid:
        return Answer("Insufficient verified evidence.", [], draft.confidence)
    return draft

```

8. Code Explanation

The wrapper refuses empty retrieval and citations outside the retrieved set, then applies an empirically tuned confidence threshold. This prevents fabricated IDs, not unsupported claims inside real documents. Production systems additionally bind claims to spans, run entailment checks, enforce tool schemas, and route high-risk or conflicting evidence to review.

9. Training / Evaluation

Create frozen, time-stamped evaluation sets with answerable, unanswerable, adversarial, multilingual, and conflicting-source cases. Measure retrieval recall@k, context precision, answer correctness, claim groundedness, citation precision/recall, abstention precision, and end-to-end task loss. Use multiple qualified annotators and adjudication. Tune prompt/retrieval/threshold on validation, not the final benchmark.

10. Complexity and Cost

RAG adds embedding/search cost and context tokens. Claim extraction plus NLI verification adds roughly one or more model calls per answer. Multi-sample self-consistency multiplies inference. Human factuality review is expensive but necessary for subtle/high-stakes claims.

11. Common Use Cases

- Enterprise RAG and knowledge assistants
- Medical/legal/financial question answering
- Search summaries and research assistants
- Autonomous agents and tool-use systems
- Code generation and API usage
- Image captioning and multimodal reports

12. Common Mistakes

- Treating low temperature as a factuality guarantee
- Evaluating only answer fluency or exact match
- Assuming a real citation supports the adjacent claim
- Mixing retrieval recall failures with generation faithfulness failures

- Forcing an answer when evidence is absent
- Letting the model self-grade without independent checks
- Using stale or contaminated reference data
- Logging/replaying confidential context without controls

13. Edge Cases / Limitations

Sources may conflict, be wrong, stale, or malicious. Some questions have evolving or subjective answers. Entailment models can hallucinate judgments too. Correct answers may use knowledge beyond supplied context, complicating faithfulness scoring. Aggressive abstention improves safety but reduces coverage and usefulness.

14. Variations

- **RAG grounding:** supplies external evidence; core AI-engineer topic.
- **Tool grounding:** uses databases/calculators/APIs; best for dynamic/exact facts.
- **Constrained decoding/structured output:** prevents format errors, not factual errors.
- **Self-consistency/reflection:** catches some instability; no truth guarantee.
- **Fine-tuning for abstention/citation:** teaches behavior but needs evidence-time controls.
- **Post-generation verification:** claim extraction, NLI, retrieval, and rule checks.

15. Related Topics

Evaluation benchmarks quantify factuality but can be contaminated. RAG improves grounding yet adds retrieval and prompt-injection surfaces. Explainability is not the same as faithful reasoning; generated chain-of-thought can itself be fabricated. Red teaming targets persuasive failure cases. Calibration and selective prediction support abstention.

16. Interview Questions

1. **What is hallucination?** Unsupported or false generated content presented as valid.
2. **Factuality versus faithfulness?** World correctness versus support by supplied evidence/instructions.
3. **Why do LLMs hallucinate?** Next-token likelihood is not a truth objective; data gaps, ambiguity, decoding, and retrieval failures contribute.
4. **Does RAG eliminate it?** No; retrieval may fail and the generator may ignore or distort evidence.
5. **How evaluate RAG hallucination?** Separate retrieval recall, grounded claims, citation support, correctness, and abstention.
6. **Does temperature zero fix it?** No; it makes decoding more deterministic, not necessarily truthful.
7. **What is citation precision?** Fraction of cited sources that actually support associated claims.
8. **How handle unknown answers?** Calibrated abstention, clarification, or human/tool escalation.
9. **Can confidence be token probability?** Usually insufficient; semantic factuality needs task calibration/evidence checks.
10. **Best mitigation?** Match claim to authoritative evidence/tool, constrain outputs, verify, and abstain when unsupported.

17. Practice Tasks

- Build a claim-level groundedness scorer over source passages.

- Compare closed-book and RAG answers on an unanswerable set.
- Diagnose retriever versus generator failures.
- Test fake citations, conflicting sources, and malicious retrieved text.
- Tune an abstention threshold using asymmetric costs.

18. Project Ideas

Project	What it does	Stack/dataset	Resume value
Citation-Verifiable RAG	Binds claims to document spans and abstains	HF, FAISS, FastAPI; public manuals	Production LLM reliability
Hallucination Eval Harness	Scores claims, citations, and unanswerables	RAGAS/custom NLI; QASPER	Evaluation engineering
Verified Data Agent	Answers numeric queries only through typed tools	Python, SQL, Pydantic	Agent/tool safety

19. Quick Revision

- **Key idea:** fluent generation optimizes likelihood, not truth.
- **Formula:** claim groundedness and risk–coverage objective.
- **Use:** any generative system making factual claims.
- **Metrics:** correctness, groundedness, citation precision/recall, abstention.
- **Trap:** RAG and low temperature reduce some errors but guarantee nothing.
- **One-liner:** Decompose retrieval, grounding, generation, and abstention before debugging hallucination.

20. Final Cheat Sheet

Item	Summary
Definition	Unsupported/false generation represented as valid
Input/output	Prompt + evidence/tools → claims with provenance or abstention
Steps	Retrieve → generate → atomize claims → verify → cite/abstain
Hyperparameters	top-k, chunking, temperature, verifier/abstention thresholds
Metrics	Retrieval recall, groundedness, correctness, citation support, coverage
Pros/cons	Grounding/verifiers reduce risk; add latency and imperfect judges
Best use	RAG, agents, high-stakes QA, research/search summaries

Evaluation Benchmarks

1. Overview

An evaluation benchmark is a standardized set of tasks, data, metrics, protocols, and reporting rules used to compare systems. Good benchmarks make a capability or risk measurable; poor ones reward memorization, shortcuts, judge preferences, or irrelevant proxy metrics. Benchmarks support research, model selection, release gates, regression testing, and vendor evaluation.

2. Intuition

A benchmark is an exam plus its grading policy. If questions leaked, grading rewards verbosity, or the exam tests arithmetic while the job needs customer judgment, a high score says little about real performance.

3. Prerequisites

- Train/dev/test methodology and statistical uncertainty
- Classification, ranking, generation, and calibration metrics
- Sampling, annotator agreement, and hypothesis testing
- LLM prompting/decoding and judge models
- Data contamination and distribution shift

4. Core Concepts

Concept	Meaning / why	Example	Interview angle
Construct validity	Benchmark measures claimed capability	Coding tasks should require code reasoning	Proxy score may not match deployment
Reliability	Results repeat across samples/runs/raters	Stable score under prompt variants	Report variance
Contamination	Test content appears in training	Memorized benchmark answer	Use private/fresh/dynamic sets
Saturation	Top models approach ceiling	99% accuracy	Benchmark stops discriminating
Ecological validity	Tasks resemble deployment	Real support tickets	Public trivia may be irrelevant
Slice coverage	Includes critical subgroups/difficulties	Languages, sites, harm categories	Macro average hides failures
Human/LLM judging	Rubric-based evaluation for open outputs	Pairwise preference	Bias, position, verbosity, self-preference
Regression suite	Stable tests for release comparison	Safety and tool-use cases	Separate from exploratory red teaming

5. Algorithm / Working Process

1. Translate system requirements and harms into observable tasks.
2. Define target population, sampling frame, slices, references, and exclusions.
3. Select task-appropriate metrics and a detailed rubric.
4. Create dev and hidden test sets; deduplicate/decontaminate and version all artifacts.

5. Standardize prompts, few-shot examples, decoding, tools, retries, and budgets.
6. Run repeated evaluation; compute uncertainty and paired comparisons.
7. Analyze errors/slices, validate judge agreement, publish limitations, and refresh for drift/saturation.

6. Mathematical Foundation

For bounded per-item scores s_i , $\bar{s} = n^{-1} \sum s_i$. A paired comparison between models uses differences $d_i = s_{Ai} - s_{Bi}$ and bootstraps items for a confidence interval. Binary accuracy standard error is approximately

$$SE = \sqrt{\hat{p}(1 - \hat{p})/n}.$$

Pass@k for c correct samples among n generated candidates is estimated as

$$\text{pass@k} = 1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}.$$

Judge agreement can use Cohen's κ ; calibration uses Brier score $n^{-1} \sum (p_i - y_i)^2$.

7. Practical Implementation

```
import numpy as np

def paired_bootstrap(a, b, repeats=5000, seed=42):
    a, b = np.asarray(a), np.asarray(b)
    assert a.shape == b.shape
    rng = np.random.default_rng(seed)
    delta = a - b
    samples = [rng.choice(delta, size=len(delta), replace=True).mean()
                for _ in range(repeats)]
    return {"mean_delta": delta.mean(),
            "ci95": np.quantile(samples, [0.025, 0.975]).tolist()}

print(paired_bootstrap([1, 0, 1, 1], [1, 0, 0, 1]))
```

8. Code Explanation

The comparison is paired because both systems answer the same items, which removes item-difficulty noise. Bootstrap resampling estimates uncertainty without assuming normal score differences. For clustered data, resample the independent unit (user/document), and correct or pre-register when making many comparisons.

9. Training / Evaluation

Never tune prompts, models, or thresholds on the reported hidden test. Record model/version, prompt hash, date, decoding, tool access, and failures/timeouts. Report mean, confidence interval, slice and worst-case metrics, latency, cost, and refusal/error rates. Calibrate LLM judges against blinded human ratings; randomize answer order and use reference/rubric evidence.

10. Complexity and Cost

Cost is $O(NK)$ model calls for N items and K repetitions/configurations, plus judge calls. Generation benchmarks can be dominated by tokens, tools, and human labeling. Parallel execution reduces wall time but not total spend. Small samples reduce cost while widening uncertainty.

11. Common Use Cases

- Comparing foundation models and prompts
- Release regression gates for RAG/agents
- Evaluating coding, reasoning, safety, factuality, and multilingual ability
- Selecting CV/NLP models for production
- Measuring robustness, fairness, privacy, and red-team defenses

12. Common Mistakes

- Benchmark shopping after seeing results
- Reporting a point estimate without confidence interval
- Ignoring prompt/model/version differences
- Letting test examples drive iterative tuning
- Treating an LLM judge as objective ground truth
- Using exact match for legitimate semantic variants
- Comparing costs/latencies under unequal budgets
- Publishing only aggregate scores and hiding refusals/timeouts

13. Edge Cases / Limitations

Public benchmarks become contaminated and saturated. Static references fail on evolving facts. Open-ended quality is multidimensional and culturally dependent. Strong benchmark performance may exploit shortcuts. Safety benchmarks can reveal attack patterns. No finite benchmark proves general capability or absence of harm.

14. Variations

- **Static public benchmarks:** reproducible but contamination-prone; placement-known.
- **Private/held-out benchmarks:** stronger selection evidence; costly maintenance.
- **Dynamic/live benchmarks:** resist memorization; lower reproducibility.
- **Human preference evaluation:** captures nuanced usefulness; expensive/noisy.
- **LLM-as-judge:** scalable; must control bias and validate agreement.
- **Simulation/task success:** best for agents/robotics; environment validity matters.
- **Adversarial benchmarks:** focus on worst cases; complement representative sets.

15. Related Topics

Data leakage includes benchmark contamination. Red teaming discovers cases that may become regression benchmarks. Hallucination needs claim/citation rubrics. Model cards report benchmark protocols and limitations. Robustness requires stress benchmarks, while governance turns selected thresholds into release gates.

16. Interview Questions

1. **What makes a good benchmark?** Valid task construct, representative sampling, reliable protocol, suitable metrics, hidden data, uncertainty, and clear limits.
2. **What is contamination?** Evaluation content or close variants influence training/tuning.
3. **Why paired bootstrap?** Models face identical items, so paired differences reduce variance.
4. **What is benchmark saturation?** Scores approach ceiling and stop distinguishing meaningful progress.
5. **Exact match limitation?** Semantically correct variants may use different strings.
6. **How validate LLM judges?** Blinded human agreement, position randomization, rubric/reference checks, multiple judges, bias tests.
7. **What should an agent benchmark include?** Task success, side effects, tool errors, recovery, latency, cost, and security.
8. **Why report slices?** Aggregate scores can hide critical languages/groups/task types.
9. **How prevent test overfitting?** Hidden/limited-access sets, dev sets, audit logs, and periodic refresh.
10. **Benchmark versus production A/B test?** Controlled capability proxy versus real user/operational outcome; both are needed.

17. Practice Tasks

- Build a paired evaluation with bootstrap intervals and slice reports.
- Compare exact match, token F1, semantic similarity, and human rubric judgments.
- Measure LLM judge position and verbosity bias.
- Detect train/test n-gram or semantic contamination.
- Convert red-team failures into versioned regression tests.

18. Project Ideas

Project	What it does	Stack/dataset	Resume value
LLM Eval Registry	Versioned prompts, runs, costs, CIs, and slices	HF, MLflow, Streamlit	LLMOps evaluation
Judge Reliability Study	Measures judge/human agreement and bias	Python; custom responses	Research methodology
RAG Release Gate	Tests retrieval, grounding, latency, and security	pytest, RAGAS/custom data	Production QA

19. Quick Revision

- **Key idea:** benchmark = tasks + data + protocol + metric + reporting.
- **Formula:** paired mean difference with bootstrap CI; pass@k for sampled code.
- **Use:** comparison, selection, regression, and release decisions.
- **Metrics:** task-specific score, CI, slice/worst-case, cost, latency.
- **Trap:** public score is not deployment validity.
- **One-liner:** Optimize for the real requirement, not the leaderboard proxy.

20. Final Cheat Sheet

Item	Summary
Definition	Standardized evidence for capability/risk comparison
Input/output	Frozen tasks/protocol + model → scores, uncertainty, errors
Steps	Define construct → sample → rubric → hide/version → run → analyze
Hyperparameters	Sample size, decoding, repetitions, judge rubric, pass k
Metrics	Mean/CI, paired delta, slice score, cost, latency, reliability
Pros/cons	Reproducible comparison; vulnerable to proxies, leakage, saturation
Best use	Model selection and controlled release regression

SHAP

1. Overview

SHAP (SHapley Additive exPlanations) assigns each feature a contribution to a prediction using Shapley values from cooperative game theory. It provides local additive explanations with desirable axioms and can aggregate them globally. SHAP is widely used for tabular models, trees, neural networks, debugging, fairness investigation, and reason-code analysis.

2. Intuition

Features are players sharing the “payout” between a baseline prediction and the current prediction. A feature receives its average marginal contribution over every order in which features could join the coalition.

3. Prerequisites

- Conditional expectation and combinatorics
- Feature attribution and model outputs/log-odds
- Correlation and causal-vs-associational reasoning
- Tree models and Python plotting
- Background/reference distributions

4. Core Concepts

Concept	Meaning / why	Example	Interview angle
Baseline ϕ_0	Expected model output over background	Average log-odds	Explanation changes with background
Local accuracy	Attributions sum to prediction difference	Base + contributions = output	Know output scale
Missingness	Absent feature receives zero in simplified representation	Masked feature	“Absent” needs a distribution

Concept	Meaning / why	Example	Interview angle
Consistency	Increased marginal effect cannot lower attribution	Axiom	Advantage over arbitrary importance
Interventional SHAP	Breaks dependencies when masking	Sample columns independently	May create impossible combinations
Conditional SHAP	Respects conditional feature distribution	Condition correlated income/role	Shares credit differently
TreeSHAP	Efficient exact/fast tree computation	XGBoost explanation	Avoid exponential enumeration
Interaction values	Pairwise contribution beyond main effects	Age × income	Costs more and needs care

5. Algorithm / Working Process

1. Choose explained output: raw score, log-odds, probability, or loss.
2. Select a representative background dataset and dependency semantics.
3. For each feature, estimate its marginal contribution over feature coalitions.
4. Return baseline plus local ϕ_j values.
5. Aggregate absolute values for global ranking and inspect direction/distribution.
6. Check stability across background samples and correlated-feature groupings.
7. Validate suspected mechanisms with held-out performance and domain/causal analysis.

6. Mathematical Foundation

For feature set F and value function $v(S)$,

$$\phi_j = \sum_{S \subseteq F \setminus \{j\}} \frac{|S|!(M - |S| - 1)!}{M!} [v(S \cup \{j\}) - v(S)].$$

Efficiency gives $f(x) = \phi_0 + \sum_j \phi_j$ on the chosen output scale. A common value function is $v(S) = E[f(X) \mid X_S = x_S]$, though implementations/assumptions vary. Exact generic computation is exponential in M ; algorithms exploit model structure or sampling.

7. Practical Implementation

```
import shap
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(n_estimators=200, random_state=42)
model.fit(X_train, y_train)

# A small representative background controls the reference distribution and cost.
background = shap.sample(X_train, 100, random_state=42)
explainer = shap.Explainer(model, background)
```

```
values = explainer(X_test.iloc[:20])
shap.plots.waterfall(values[0, :, 1]) # class-1 explanation; inspect API shape
```

8. Code Explanation

`shap.Explainer` selects a suitable algorithm from the model/masker. The background defines what “missing” features are compared with. The returned tensor shape depends on SHAP/model versions and multiclass output, so inspect `values.shape` before indexing. Waterfall plots show how each contribution moves the baseline toward one prediction.

9. Training / Evaluation

Compute explanations on held-out representative slices. Test local additivity numerically, repeat with several backgrounds, perturb nearby points, and compare correlated-feature groupings. Evaluate whether top attributions identify injected signal/leakage. For user-facing reason codes, test stability, faithfulness, privacy, and actionable wording rather than relying on attractive plots.

10. Complexity and Cost

Generic exact Shapley enumeration is $O(2^M)$. KernelSHAP uses sampled coalitions and repeated model calls; TreeSHAP is polynomial and typically fast for tree ensembles. Deep approximations require model passes. Background size, samples, features, outputs, and rows determine memory/time.

11. Common Use Cases

- Local credit/fraud/churn explanations
- Global feature effect and interaction analysis
- Finding leakage and spurious proxies
- Comparing subgroup attribution patterns
- Explaining tree ensembles in regulated workflows
- Investigating neural tabular/CV/NLP models with suitable explainers

12. Common Mistakes

- Saying positive SHAP means feature value is positively correlated with target globally
- Ignoring whether values explain probability, logit, or raw score
- Choosing an unrepresentative background
- Treating SHAP as causal
- Ranking only mean absolute SHAP and losing direction/subgroups
- Trusting attributions under strong correlation without sensitivity analysis
- Comparing SHAP magnitudes from incompatible model/output setups
- Exposing sensitive information through local explanations

13. Edge Cases / Limitations

Correlated features make “missing feature” semantics ambiguous. Conditional estimation is difficult in high dimensions; interventional masking can create impossible samples. Explanations can be unstable off

distribution. SHAP faithfully attributes model behavior under assumptions, not data truth or fairness. Large feature sets and generative models are costly.

14. Variations

- **TreeSHAP:** tree ensembles; most placement/project relevant.
- **KernelSHAP:** model-agnostic but slow.
- **LinearSHAP:** efficient linear models with dependency options.
- **DeepSHAP/GradientSHAP:** approximate neural attribution.
- **PartitionSHAP:** hierarchical feature coalitions; text/images/correlated groups.
- **SHAP interaction values:** decomposes pair effects; advanced analysis.

15. Related Topics

LIME fits a sparse local surrogate, while SHAP derives an additive allocation with Shapley weighting. Permutation importance measures global performance loss. PDP/ALE show average response shapes. Explainability supplies validation criteria; fairness analysis can use SHAP to find proxies but needs group outcome/error metrics.

16. Interview Questions

1. **What is SHAP?** Shapley-value feature attribution for model outputs.
2. **Core intuition?** Average each feature's marginal contribution across all coalitions/orders.
3. **What is baseline?** Expected model output under the chosen background.
4. **What is local accuracy?** Baseline plus attributions equals explained output.
5. **Why is exact SHAP expensive?** It considers exponentially many coalitions.
6. **TreeSHAP advantage?** Exploits tree structure for efficient exact/fast attribution.
7. **Conditional versus interventional?** Preserve dependencies versus break them during masking.
8. **Does SHAP imply causality?** No; it attributes model associations under a value-function assumption.
9. **How handle correlated features?** Group them, use conditional approaches, and report background/dependency sensitivity.
10. **SHAP versus feature importance?** SHAP is local and signed, aggregatable; impurity importance is global, unsigned, and biased.

17. Practice Tasks

- Verify SHAP additivity for a tree model.
- Compare backgrounds from training, a subgroup, and k-means summaries.
- Show attribution sharing with two correlated duplicate features.
- Find an injected leakage column with SHAP.
- Compare SHAP global rankings with permutation importance.

18. Project Ideas

Project	What it does	Stack/dataset	Resume value
SHAP Stability Auditor	Measures background/seed/correlation sensitivity	SHAP, sklearn; Adult	XAI rigor
Fraud Reason-Code API	Returns controlled local reasons and monitoring	XGBoost, SHAP, FastAPI	Deployable explainability
Interaction Explorer	Visualizes pair effects and subgroup differences	LightGBM, Streamlit; lending data	Advanced tabular analysis

19. Quick Revision

- **Key idea:** distribute prediction-minus-baseline using average marginal contributions.
- **Formula:** weighted coalition marginal-contribution sum.
- **Use:** local/global attribution, especially tree models.
- **Metrics:** additivity, stability, fidelity sanity checks, runtime.
- **Trap:** baseline/correlation/output scale control meaning.
- **One-liner:** SHAP explains a model relative to a background, not reality causally.

20. Final Cheat Sheet

Item	Summary
Definition	Shapley-based additive feature contribution
Input/output	Model + row + background → baseline and ϕ_j values
Steps	Choose output/background → mask coalitions → marginal contributions → aggregate
Hyperparameters	Background, masker/dependency, algorithm, coalition samples
Metrics	Additivity, stability, usefulness, compute
Pros/cons	Axiomatic and local/global; costly and correlation-sensitive
Best use	Tree/tabular explanations, debugging, reason analysis

LIME

1. Overview

LIME (Local Interpretable Model-agnostic Explanations) explains one prediction by sampling perturbed inputs near it, querying the black-box model, weighting samples by proximity, and fitting a sparse interpretable surrogate. It works with tabular, text, and image models and is valuable when model internals are unavailable.

2. Intuition

To understand a complicated road near one junction, approximate only that neighborhood with a straight line. The line need not describe the entire city; its usefulness depends on whether sampled nearby points are

realistic and the black box behaves smoothly there.

3. Prerequisites

- Weighted linear regression and regularization
- Distance metrics and kernels
- Feature perturbation/masking
- Local versus global explanations
- Model probability APIs

4. Core Concepts

Concept	Meaning / why	Example	Interview angle
Interpretable representation	Human-readable binary/binning features	Word present, superpixel on	May differ from raw input
Perturbation	Synthetic neighborhood samples	Remove words or gray superpixels	Realism determines fidelity
Proximity kernel	Gives near samples more weight	Exponential of distance	Kernel width changes explanation
Local surrogate	Simple model approximating black box nearby	Sparse linear model	Not the original model
Sparsity	Shows limited features	Top five words	Comprehension vs fidelity
Local fidelity	Agreement near explained point	Weighted R^2	Must be reported/checked
Instability	Seeds/samples can alter features	Different top words	Repeat and assess overlap

5. Algorithm / Working Process

1. Convert instance x to interpretable representation x' .
2. Sample perturbed representations $z'_1, \dots, z'_N$.
3. Map them back to model inputs and query $f(z_i)$.
4. Weight each by proximity $\pi_x(z_i)$.
5. Fit a sparse simple model minimizing weighted error plus complexity.
6. Return the strongest signed surrogate coefficients.
7. Repeat seeds/kernel widths and report fidelity/stability.

6. Mathematical Foundation

LIME chooses

$$\xi(x) = \arg \min_{g \in G} \sum_i \pi_x(z_i) (f(z_i) - g(z'_i))^2 + \Omega(g),$$

where G is interpretable models and Ω penalizes complexity. A common kernel is

$$\pi_x(z) = \exp(-D(x, z)^2/\sigma^2).$$

Small σ is more local but yields fewer effective samples; large σ improves stability while risking a poor approximation of nonlinear behavior.

7. Practical Implementation

```
from lime.lime_tabular import LimeTabularExplainer

explainer = LimeTabularExplainer(
    X_train.to_numpy(),
    feature_names=X_train.columns.tolist(),
    class_names=["negative", "positive"],
    mode="classification",
    discretize_continuous=True,
    random_state=42,
)
explanation = explainer.explain_instance(
    X_test.iloc[0].to_numpy(), model.predict_proba,
    labels=(1,), num_features=8, num_samples=5000,
)
print(explanation.as_list(label=1))
```

8. Code Explanation

The explainer learns perturbation statistics from training features. `predict_proba` is the black-box query. `num_samples` controls Monte Carlo fidelity/cost; `num_features` controls explanation sparsity. Discretization produces readable intervals but can create boundary artifacts. Fixed seeds reproduce one run, not stability—rerun multiple seeds.

9. Training / Evaluation

LIME itself is fitted per explanation. Evaluate weighted local R^2 /error, top-feature overlap across seeds, sign consistency, sensitivity to kernel width/background, and perturbation validity. Test explanations on held-out representative cases and known synthetic functions. Do not use final test explanations to tune the underlying model repeatedly.

10. Complexity and Cost

Per explanation, cost is N black-box queries plus fitting a weighted sparse model, roughly $O(Nd^2)$ for naive linear algebra and less with efficient solvers/sparse features. Text/image queries dominate. LIME is naturally parallel across instances.

11. Common Use Cases

- Explaining proprietary/remote model APIs
- Token-level text classification explanations

- Superpixel explanations for image classifiers
- Local debugging of tabular predictions
- Comparing local behavior across model versions
- Producing candidate reason codes subject to validation

12. Common Mistakes

- Presenting surrogate coefficients as original-model parameters
- Ignoring low local fidelity
- Sampling impossible tabular combinations or unnatural masked text/images
- Choosing one seed and assuming stability
- Interpreting discretized-bin boundaries literally
- Using default distance/kernel for every modality
- Treating LIME coefficients as causal
- Generalizing one local explanation globally

13. Edge Cases / Limitations

Highly nonlinear/discontinuous neighborhoods are poorly approximated by a line. High dimensions make neighborhoods sparse. Correlated constraints are broken by independent perturbation. Image superpixels and word deletion may be out of distribution. Explanations can be gamed by models that behave differently on perturbed versus natural inputs.

14. Variations

- **Tabular LIME:** perturbs binned/numeric features; placement-important.
- **Text LIME:** word presence masks; common NLP demo.
- **Image LIME:** superpixel on/off masks; common CV demo.
- **SP-LIME:** selects representative explanations for global coverage.
- **Anchors:** high-precision if-then local rules; easier to communicate.
- **Manifold-aware LIME:** generates plausible neighbors; research extension.

15. Related Topics

SHAP uses Shapley coalition weighting and an additive consistency framework; LIME directly optimizes local surrogate fidelity/simplicity. Counterfactual explanations search for outcome-changing instances. Local linear gradients explain infinitesimal behavior with model access. Explainability defines audience and validation requirements.

16. Interview Questions

1. **What is LIME?** A model-agnostic local surrogate explanation method.
2. **Main steps?** Perturb, query, proximity-weight, fit sparse surrogate, report coefficients.
3. **Why model-agnostic?** It needs only prediction queries.
4. **What does kernel width do?** Controls locality versus effective sample size/stability.
5. **What is local fidelity?** Surrogate agreement with black-box predictions near the instance.
6. **Why unstable?** Random sampling, collinearity, sparse neighborhoods, and nonlinear boundaries.

7. **LIME versus SHAP?** LIME optimizes a local surrogate; SHAP targets Shapley additive allocations with specific weighting/axioms.
8. **How improve reliability?** Plausible perturbations, more samples, repeated seeds, tuned distance, report fidelity.
9. **Can LIME explain causality?** No; it approximates model response to generated perturbations.
10. **What can adversarially fool LIME?** A model can detect synthetic perturbations and expose benign behavior only there.

17. Practice Tasks

- Implement LIME's weighted local linear regression from scratch.
- Plot fidelity/stability versus samples and kernel width.
- Compare independent and correlation-aware perturbations.
- Explain the same instance with LIME and SHAP.
- Debug an image explanation by changing superpixel segmentation.

18. Project Ideas

Project	What it does	Stack/dataset	Resume value
LIME Reliability Dashboard	Visualizes fidelity and seed/kernel sensitivity	LIME, sklearn, Streamlit	Responsible XAI practice
API Black-Box Explainer	Explains predictions with query budgets/caching	FastAPI, LIME; any hosted model	Model-agnostic engineering
Manifold LIME Study	Generates neighbors with an autoencoder	PyTorch; tabular/image dataset	Research extension

19. Quick Revision

- **Key idea:** locally approximate a black box using proximity-weighted synthetic samples.
- **Formula:** weighted fidelity loss plus surrogate complexity.
- **Use:** local explanations when only query access exists.
- **Metrics:** local R^2 , stability, sparsity, perturbation plausibility.
- **Trap:** explanation is the surrogate, not the black box itself.
- **One-liner:** A LIME explanation is credible only where its sampled neighborhood is valid and its fidelity is high.

20. Final Cheat Sheet

Item	Summary
Definition	Local sparse surrogate fitted to weighted perturbations
Input/output	Instance + prediction API → signed local feature weights
Steps	Represent → perturb → query → weight → fit → validate
Hyperparameters	Samples, kernel width, distance, feature count, discretization

Item	Summary
Metrics	Weighted fidelity, stability, sparsity, runtime
Pros/cons	Model-agnostic/intuitive; unstable and perturbation-sensitive
Best use	Local black-box tabular, text, and image explanation

Model Cards

1. Overview

A model card is a versioned document describing a model's purpose, owners, training/evaluation context, performance, limitations, ethical/security risks, and operational requirements. It supports informed model selection, review, deployment, audits, and incident response. It is evidence-backed communication, not a marketing page or substitute for testing.

2. Intuition

A model card is a medicine label plus engineering datasheet: what it is for, who should not use it, what evidence supports it, known side effects, operating conditions, and whom to contact.

3. Prerequisites

- ML lifecycle, metrics, slices, and uncertainty
- Dataset lineage and intended use
- Risk assessment, privacy/security, and governance
- Model/version registry and deployment monitoring
- Clear technical writing

4. Core Concepts

Concept	Meaning / why	Example	Interview angle
Model details	Name, version, architecture, owner, license	fraud-v3.2	Trace artifact exactly
Intended use	Supported users/tasks/context	Triage, not autonomous denial	Include out-of-scope uses
Training context	Data snapshot, objective, dependencies	Data through 2025-Q4	Link dataset card
Evaluation	Protocol, metrics, slices, CIs	Recall per region	Reproducibility over headline score
Limitations	Known failure modes and uncertainty	Weak on scans from device X	Specific and actionable

Concept	Meaning / why	Example	Interview angle
Risk/mitigation	Fairness, privacy, misuse, security controls	Human review on low confidence	Residual risk remains
Operations	Thresholds, monitoring, rollback, expiry	Drift alert and owner	Living document
Change history	Differences and approvals	v3 changed tokenizer	Enables governance/audit

5. Algorithm / Working Process

1. Identify artifact, owner, decision, users, stakeholders, and release stage.
2. Link immutable code/data/config/checkpoint versions.
3. State intended, unsupported, and prohibited uses.
4. Record training objective/data context without exposing secrets or PII.
5. Report reproducible evaluation, slices, uncertainty, robustness, fairness, privacy, and security tests.
6. Describe limitations, mitigations, human oversight, monitoring, rollback, and contact.
7. Review with responsible owners and update on every material model/data/use change.

6. Mathematical Foundation

Model cards do not introduce a training formula; they report estimands with uncertainty. For metric m , record $\hat{m}$, confidence interval, sample size, threshold, and slice. For a release gate, an example requirement is

$$LCB_{95\%}(Recall_{critical}) \geq r_{min}, \quad \max_g FNR_g - \min_g FNR_g \leq \epsilon.$$

The lower confidence bound avoids approving a critical model on an uncertain point estimate. Report calibration and operating threshold rather than AUROC alone.

7. Practical Implementation

```
from pathlib import Path
import json

def render_model_card(metadata: dict) -> str:
    required = {"name", "version", "owner", "intended_use", "limitations",
"metrics"}
    missing = required - metadata.keys()
    if missing:
        raise ValueError(f"Missing model-card fields: {sorted(missing)}")
    return f"""# {metadata['name']} ({metadata['version']})
Owner: {metadata['owner']}

## Intended use
{metadata['intended_use']}

## Evaluation
```json
{json.dumps(metadata['metrics'], indent=2)}
```
```

```
## Limitations
{metadata['limitations']}
"""

# Path("MODEL_CARD.md").write_text(render_model_card(metadata), encoding="utf-8")
```

8. Code Explanation

The small validator fails release documentation when critical fields are absent and renders machine-held metadata into readable Markdown. In a real registry, generate values from immutable evaluation artifacts, review narrative fields, sign approvals, and avoid hand-copying scores that can drift from the released checkpoint.

9. Training / Evaluation

Document dataset and time/site/entity splits, baseline, metric definitions, thresholds, CIs, group supports, calibration, robustness, privacy and red-team results. Include failures and abstentions. The card itself should pass freshness, completeness, link-integrity, and artifact-consistency checks during release.

10. Complexity and Cost

Rendering is trivial; producing credible evidence requires evaluation runs, specialist reviews, and maintenance. Automation reduces clerical drift but cannot decide intended use or acceptable residual risk. Store concise summaries with links to detailed artifacts.

11. Common Use Cases

- Foundation-model releases and APIs
- Internal registry approval and handoff
- Regulated credit/health/hiring models
- Vendor model assessment
- Edge-device and CV model deployment
- Incident investigation and rollback

12. Common Mistakes

- Copying a template with vague “may be biased” language
- Omitting data/version/prompt/threshold details
- Reporting averages without slices or uncertainty
- Listing mitigations without evidence they work
- Treating the card as permanent after deployment drift
- Hiding poor results or unsupported uses
- Publishing sensitive training or security details
- Confusing compliance documentation with approval

13. Edge Cases / Limitations

Cards become stale, depend on self-reporting, and cannot enumerate unknown failure modes. Different deployments can invalidate one card. Public transparency may conflict with privacy, IP, or security; provide tiered internal/public views while preserving decision-relevant information.

14. Variations

- **Public model card:** external transparency; omit exploit-enabling secrets.
- **Internal system card:** richer lineage, controls, approvals, incidents.
- **Service/API card:** includes versioning, quotas, data use, SLAs.
- **Generative-AI system card:** broad safety evaluations and mitigations.
- **Automated registry report:** machine-generated metrics plus reviewed narrative.

15. Related Topics

Dataset cards document data; model cards document learned artifacts and evaluated behavior. AI governance uses cards as approval evidence. Evaluation benchmarks supply results. Red-team reports, privacy reviews, fairness analyses, and monitoring plans should link from the card.

16. Interview Questions

1. **What is a model card?** Versioned documentation of intended use, evidence, limitations, risks, and operation.
2. **Why useful?** It enables informed selection, review, accountability, and safe deployment.
3. **What must it identify?** Exact artifact/version, owner, data context, use, evaluation, risks, and controls.
4. **Card versus model registry?** Card is human-readable evidence; registry manages artifacts/metadata/lifecycle.
5. **Should it include failures?** Yes, representative failures and unsupported contexts are essential.
6. **How keep current?** Generate metrics from registry, assign owner/expiry, update on material change.
7. **Can it prove safety?** No; it records scoped evidence and residual risk.
8. **What about slices?** Report support, metrics, and uncertainty for relevant populations/conditions.
9. **Public versus internal?** Different detail/access, consistent core claims.
10. **When block release?** Missing ownership/evidence, failed gate, undocumented high risk, or stale evaluation.

17. Practice Tasks

- Write a card for an existing classifier with three concrete limitations.
- Generate metric tables directly from JSON artifacts.
- Add freshness and required-field CI checks.
- Review a vague card and identify unverifiable claims.
- Extend a card for a RAG model with retrieval/security evaluation.

18. Project Ideas

| Project | What it does | Stack/dataset | Resume value |
|---------------|--|--------------------------------|-----------------------|
| Model Card CI | Builds cards and blocks missing/stale evidence | Python, MLflow, GitHub Actions | Governance automation |

| Project | What it does | Stack/dataset | Resume value |
|-------------------|--|------------------------|-----------------------------|
| Card Catalog | Searchable version/risk/evaluation portal | FastAPI, SQLite, React | Internal ML platform |
| GenAI System Card | Documents RAG, red-team, cost, and grounding | HF/RAG evaluation | Responsible GenAI portfolio |

19. Quick Revision

- **Key idea:** versioned, scoped evidence for responsible model use.
- **Formula:** report estimate + CI + support + threshold; use lower-bound gates.
- **Use:** release, registry, procurement, audit, incidents.
- **Metrics:** completeness/freshness plus task/risk metrics.
- **Trap:** a polished card without reproducible evidence is marketing.
- **One-liner:** A model card tells the next decision-maker what the model can and cannot safely support.

20. Final Cheat Sheet

| Item | Summary |
|-----------------|---|
| Definition | Living, versioned model evidence and use documentation |
| Input/output | Artifact + evaluations + decisions → reviewed card |
| Steps | Identify → scope → link lineage → report → limit → operate → update |
| Hyperparameters | Thresholds, release gates, expiry, slices (not training parameters) |
| Metrics | Task/slice/robustness/fairness/privacy plus card freshness |
| Pros/cons | Improves handoff/accountability; can become stale or performative |
| Best use | Every production/released model, especially high impact |

Dataset Cards

1. Overview

A dataset card documents why a dataset exists, how it was collected, composed, processed, labeled, governed, and what uses or harms are known. It helps engineers decide whether data is fit for purpose, reproduce preparation, trace consent/licensing, and interpret model performance.

2. Intuition

A dataset without documentation is a box of ingredients without labels, origin, allergens, or expiry. It may look usable but you cannot responsibly serve it.

3. Prerequisites

- Sampling, labeling, missing data, and bias

- Data lineage/versioning and train/test splitting
- Consent, licensing, privacy, and security basics
- Descriptive statistics and slice analysis
- Domain knowledge of the measured construct

4. Core Concepts

| Concept | Meaning / why | Example | Interview angle |
|---------------|--|-----------------------------|-----------------------------|
| Motivation | Purpose, funder, creators | Speech recognition research | Incentives shape collection |
| Composition | Instances, fields, labels, groups, missingness | 10k clips, 8 languages | Counts and distributions |
| Collection | Source, period, sampling, instruments | Opt-in mobile recordings | Selection/measurement bias |
| Annotation | Instructions, raters, agreement, adjudication | Three clinicians per image | Label uncertainty |
| Preprocessing | Filtering, dedup, normalization, splits | Speaker-disjoint split | Leakage prevention |
| Uses | Suitable/unsuitable tasks/populations | Not for identity inference | Scope limitations |
| Governance | License, consent, access, retention, deletion | Restricted research license | Authority to use |
| Maintenance | Owner, version, changes, issue channel | v2 removes duplicates | Living asset |

5. Algorithm / Working Process

1. Name owner, purpose, unit of observation, target population, and construct.
2. Record source, inclusion/exclusion, time/geography, consent, license, and compensation.
3. Describe schema, distributions, missingness, sensitive fields, and representative examples.
4. Document annotation instructions, expertise, disagreement, and quality control.
5. Record transformations, deduplication, split keys, and leakage checks as executable lineage.
6. Assess bias, privacy, safety, misuse, and downstream limitations.
7. Assign version, checksum, access tier, retention/deletion, maintainer, and change log.

6. Mathematical Foundation

Document sample support n_g , proportion $\hat{p}_g = n_g/n$, uncertainty, missingness $P(M_j = 1 \mid A, Y)$, label prevalence, and annotator agreement. Cohen's kappa is

$$\kappa = \frac{p_o - p_e}{1 - p_e}.$$

For dataset shift, compare distributions with appropriate distances (e.g., PSI, Wasserstein, or MMD) but interpret them using domain thresholds. A checksum $H(file)$ establishes artifact identity, not quality.

7. Practical Implementation

```
import hashlib, json
from pathlib import Path

def dataset_manifest(csv_path, *, version, license_name, split_key):
    path = Path(csv_path)
    digest = hashlib.sha256(path.read_bytes()).hexdigest()
    return {
        "name": path.stem, "version": version, "sha256": digest,
        "license": license_name, "split_key": split_key,
        "size_bytes": path.stat().st_size,
        "limitations": [], "maintainer": "REQUIRED",
    }

# manifest = dataset_manifest("train.csv", version="1.0", license_name="...",
# split_key="user_id")
# print(json.dumps(manifest, indent=2))
```

8. Code Explanation

The manifest binds documentation to exact bytes and records the independent split key. Hashes detect accidental replacement but do not prove provenance or integrity against a malicious publisher unless signed/trusted. Narrative collection, consent, population, annotation, risk, and limitation fields still require reviewed evidence.

9. Training / Evaluation

Validate schema/ranges, duplicate and entity overlap, label distribution, missingness, group coverage, temporal/geographic coverage, agreement, and contamination. Report train/validation/test creation before model results. Create automated data tests, but retain human review for construct validity and consent.

10. Complexity and Cost

Basic profiling is $O(nd)$; exact duplicate hashing is linear, semantic duplication and PII scans cost more. Annotation audits and consent remediation dominate. Profiles of huge datasets should stream/sample while preserving rare-group checks.

11. Common Use Cases

- Public research dataset releases
- Enterprise feature/training-data catalogs
- Foundation-model corpora and fine-tuning sets
- Medical and biometric collections
- Synthetic dataset releases

- Procurement of third-party data

12. Common Mistakes

- Listing columns without collection context
- “Publicly available” treated as consent/license
- Omitting removed/excluded populations
- Reporting totals without intersections or missingness
- Documenting current files but not transformations/splits
- Assuming synthetic data is private/unbiased
- No owner, version, checksum, retention, or deletion path
- Copying a card across versions without re-audit

13. Edge Cases / Limitations

Provenance may be unknowable for web-scale corpora. Rich composition statistics can create privacy risk. Licenses and consent can conflict across sources. Dataset behavior depends on downstream task, so a card cannot certify universal suitability. Data evolves while models retain older influence.

14. Variations

- **Datasheets for datasets:** comprehensive lifecycle documentation.
- **Data statements:** NLP-focused population/linguistic context.
- **Data nutrition labels:** compact standardized summaries.
- **Hugging Face dataset cards:** repository-integrated Markdown/YAML.
- **Internal manifests:** machine-readable lineage/access/deletion controls.
- **Synthetic-data cards:** include generator, source leakage, and fidelity/privacy tests.

15. Related Topics

Bias audits populate representation and label limitations. Privacy/PII handling governs access and release. Data leakage determines valid splits and contamination. Model cards link to dataset versions but add learned behavior. Governance assigns ownership and retention.

16. Interview Questions

1. **What is a dataset card?** Versioned documentation of dataset origin, composition, processing, governance, uses, and limits.
2. **Why not only schema?** Meaning depends on population, collection, measurement, labels, consent, and exclusions.
3. **What annotation details matter?** Instructions, rater expertise/demographics, agreement, adjudication, uncertainty.
4. **How document splits?** Exact keys, time cutoff, random seed/code, duplicate/leakage checks.
5. **What is construct validity?** Whether recorded variables/labels measure the intended concept.
6. **Public data safe to use?** Not automatically; privacy, terms, consent, and harm still matter.
7. **How version?** Immutable snapshot/checksum plus semantic change log and lineage.
8. **Synthetic data safe?** Not automatically; test memorization, privacy, bias, and task fidelity.
9. **Card versus model card?** Dataset artifact versus learned model behavior/use.

10. **Who owns it?** Named maintainer with authority for access, changes, issues, and deletion.

17. Practice Tasks

- Create a dataset card for Adult or a project dataset.
- Detect entity/near-duplicate leakage and update limitations.
- Measure group support, missingness, and label agreement.
- Trace one raw field into a model feature.
- Design deletion propagation across snapshots/features/models.

18. Project Ideas

| Project | What it does | Stack/dataset | Resume value |
|---------------------------|---|--------------------------------|----------------------------|
| Dataset Card Generator | Profiles data and merges reviewed narrative | Pandas, Great Expectations | Data-centric MLOps |
| Corpus Provenance Catalog | Tracks URL/license/dedup/PII status | DuckDB, MinHash; public text | Foundation-data governance |
| Label Quality Portal | Shows disagreement, slices, adjudication | Streamlit; crowdsourced labels | Annotation engineering |

19. Quick Revision

- **Key idea:** make dataset origin, meaning, suitability, and governance inspectable.
- **Formula:** group/missingness/agreement statistics plus immutable checksum.
- **Use:** dataset creation, selection, sharing, and audit.
- **Metrics:** support, prevalence, missingness, agreement, duplicates, drift.
- **Trap:** schema and license name alone are not provenance or fitness.
- **One-liner:** Dataset cards turn undocumented assumptions into reviewable evidence.

20. Final Cheat Sheet

| Item | Summary |
|-----------------|--|
| Definition | Lifecycle documentation for a versioned dataset |
| Input/output | Data + provenance + governance → card/manifest |
| Steps | Purpose → collect → compose → label → process/split → risk → maintain |
| Hyperparameters | Slice granularity, duplicate threshold, split seed/cutoff |
| Metrics | Counts, missingness, agreement, overlap, distribution distances |
| Pros/cons | Improves fitness/reproducibility; relies on honest maintained provenance |
| Best use | Every shared/training dataset, especially sensitive or public data |

Red Teaming

1. Overview

AI red teaming is authorized adversarial testing that attempts to make a model or system violate security, safety, privacy, reliability, or misuse requirements. It covers more than prompts: identity, APIs, tools, RAG sources, permissions, memory, orchestration, UI, monitoring, and human workflows. Its output is reproducible evidence and remediation priorities, not a collection of sensational chats.

2. Intuition

Quality assurance checks whether a door opens normally; a red team thinks like a motivated intruder, confused user, insider, or malicious document and tests locks, windows, key handling, alarms, and response.

3. Prerequisites

- Threat modeling, trust boundaries, and abuse cases
- LLM/RAG/agent architectures
- Prompt injection, privacy, authorization, and tool security
- Responsible testing, scope, and evidence handling
- Severity scoring and incident response

4. Core Concepts

| Concept | Meaning / why | Example | Interview angle |
|---------------------|--|---------------------------------|---------------------------------|
| Rules of engagement | Authorized scope, accounts, data, timing, stop rules | Staging only, no real users | Essential ethical boundary |
| Threat actor | Capability, access, motive | Anonymous user vs insider | Findings depend on actor |
| Attack surface | Every input/action boundary | Uploads, retrieval, tools, logs | Test system, not just model |
| Abuse case | Attacker goal and harm | Exfiltrate private document | Outcome-focused |
| Adaptive attack | Changes based on defenses/output | Encoded multi-turn payload | Static lists are insufficient |
| Evidence | Reproducible request/state/output | Trace and artifact IDs | Minimize sensitive content |
| Severity | Impact × likelihood/exploitability | Cross-tenant read is critical | Prioritize real harm |
| Purple teaming | Red and defensive teams iterate | Retest controls together | Converts findings to resilience |

5. Algorithm / Working Process

1. Obtain written authorization and define scope, safety, data handling, and escalation.
2. Map assets, actors, trust boundaries, data flows, tools, and existing controls.

3. Create abuse cases across confidentiality, integrity, availability, misuse, fairness, and reliability.
4. Run manual, automated, and adaptive tests with controlled identities/query budgets.
5. Record minimal reproducible evidence, preconditions, impact, and control failures.
6. Triage severity, identify root cause, assign remediation and owner.
7. Retest fixes and add safe regression cases/monitoring without publishing harmful details unnecessarily.

6. Mathematical Foundation

Red teaming is risk-driven rather than one algorithm. A simple priority score is

$$Risk = Impact \times Likelihood,$$

refined by exploitability, exposure, detectability, and affected population. Attack success rate is $ASR = s/N$, with a binomial confidence interval. Track conditional ASR by attack family and attacker capability. Defense utility includes false-block rate, because a control that blocks all users is “secure” but useless.

7. Practical Implementation

```
from dataclasses import dataclass

@dataclass(frozen=True)
class RedTeamCase:
    case_id: str
    category: str
    prompt: str
    should_block: bool

def run_cases(cases, system_call, policy_check):
    results = []
    for case in cases:
        response = system_call(case.prompt) # use an authorized test environment
        violation = policy_check(case, response)
        results.append({"id": case.case_id, "category": case.category,
                       "violation": violation})
    return results
```

8. Code Explanation

Cases have stable IDs and expected security behavior, enabling regression testing. The system call must include the real prompt assembly, retrieval, tools, and permissions. `policy_check` should combine deterministic checks and reviewed rubrics; do not let the same model be the only attacker and judge. Store hashes/redacted evidence where outputs are sensitive.

9. Training / Evaluation

Measure ASR by category/capability, harm severity, false refusals, detection rate, time-to-detect, and remediation/retest status. Separate exploratory discoveries from frozen regression suites. Test base model and end-to-end deployment. Use blinded human adjudication for ambiguous harms and control for prompt/judge variance.

10. Complexity and Cost

Automated suites cost cases × variants × model/tool calls. Human creative testing, domain experts, infrastructure isolation, and remediation dominate. Agent attacks can create external side effects; use sandboxed mock tools and strict budgets.

11. Common Use Cases

- LLM safety and policy bypass testing
- RAG prompt injection and cross-tenant access
- Agent tool misuse and excessive permissions
- PII/model extraction
- Misinformation, bias, and high-stakes reliability
- Pre-release and post-incident validation

12. Common Mistakes

- Testing without explicit authorization or stop rules
- Testing only direct jailbreak phrases
- Ignoring auth, tools, retrieval, memory, and UI
- Reporting examples without root cause or reproducibility
- Using ASR without severity or false-positive cost
- Leaking exploit details or sensitive outputs
- "Fixing" with a blacklist and no adaptive retest
- Letting red-team data contaminate final evaluation

13. Edge Cases / Limitations

No test proves absence of vulnerabilities. Stochastic models complicate reproduction. A realistic attack may be unsafe to run against production. Cultural harms require diverse expertise. Defense changes create new bypasses and utility regressions. Results age quickly as models, prompts, tools, and attackers change.

14. Variations

- **Manual creative red team:** discovers novel chains; high skill/cost.
- **Automated adversarial generation:** scales variants; judge quality limits it.
- **Domain red team:** medicine, bio, cyber, elections; specialist essential.
- **Privacy/security red team:** extraction, auth, injection, exfiltration.
- **Purple team:** collaborative attack/defense iteration; production best practice.
- **Bug bounty/external testing:** diverse attackers under controlled disclosure.

15. Related Topics

Prompt injection and jailbreaks are specific attack families. Evaluation benchmarks provide stable coverage; red teaming explores unknowns. Formal verification proves narrow properties, while red teams test broader reality. Governance authorizes scope, accepts residual risk, and tracks remediation.

16. Interview Questions

1. **What is AI red teaming?** Authorized adversarial testing of end-to-end AI risks and controls.
2. **How differs from benchmark evaluation?** Red teaming adaptively searches failures; benchmarks measure fixed representative/regression tasks.
3. **First step?** Written scope/rules of engagement and threat model.
4. **Why end-to-end?** Vulnerabilities often live in retrieval, tools, auth, memory, or integration.
5. **What is ASR?** Successful attacks divided by attempts under a declared protocol.
6. **How prioritize findings?** Impact, likelihood/exploitability, affected assets/users, control coverage.
7. **What evidence record?** Minimal reproducible inputs, state, version, trace, outcome, and impact.
8. **How validate fix?** Root-cause control, adaptive retest, frozen regression, utility check.
9. **Can LLM judges suffice?** No; validate against human/rule evidence and avoid shared failure modes.
10. **What is purple teaming?** Attackers and defenders collaborate to improve detection and controls.

17. Practice Tasks

- Threat-model a document-answering agent.
- Create an authorized regression suite across five attack categories.
- Score findings by impact/likelihood and justify priorities.
- Diagnose a blacklist defense and design adaptive variants.
- Write a minimal, redacted finding report with retest criteria.

18. Project Ideas

| Project | What it does | Stack/dataset | Resume value |
|----------------------------|--|---------------------------------------|-------------------------|
| Agent Red-Team Harness | Tests mock tools, permissions, injection, exfiltration | Python, promptfoo/garak, sandbox APIs | AI security engineering |
| RAG Attack Lab | Injects malicious docs and measures control ASR | LangChain/LlamaIndex, FAISS | RAG security depth |
| Safety Regression Registry | Versions cases, evidence, severity, and fixes | pytest, MLflow, dashboard | Secure LLMOps |

19. Quick Revision

- **Key idea:** authorized, threat-driven search for end-to-end failure.
- **Formula:** conditional ASR plus impact/likelihood severity.
- **Use:** pre-release, major change, high-risk system, incident response.
- **Metrics:** ASR, severity, false block, detection, remediation time.
- **Trap:** a list of naughty prompts is not a system red team.
- **One-liner:** Red team the trust boundaries and consequences, not just the chatbot text.

20. Final Cheat Sheet

| Item | Summary |
|--------------|---|
| Definition | Authorized adversarial system-risk assessment |
| Input/output | Scope + threat model + system → findings, evidence, regressions |

| Item | Summary |
|-----------------|---|
| Steps | Authorize → map → attack → evidence → triage → fix → retest |
| Hyperparameters | Query budget, actor access, variants, severity thresholds |
| Metrics | ASR, impact, detection, false blocks, closure/retest rate |
| Pros/cons | Finds realistic chains; incomplete, costly, and rapidly aging |
| Best use | LLM/RAG/agent releases and high-impact ML systems |

Prompt Injection

1. Overview

Prompt injection occurs when untrusted content influences an LLM to override intended instructions or misuse connected data/tools. Direct injection comes from a user; indirect injection is embedded in webpages, emails, documents, images, tool output, or retrieved passages. Because LLMs process instructions and data in the same token stream, prompt hierarchy alone is not a security boundary.

2. Intuition

It resembles SQL injection at the trust-boundary level—data is treated as control—but parameterized SQL has no exact LLM equivalent. Therefore secure design limits authority and validates actions instead of relying on the model to “ignore malicious instructions.”

3. Prerequisites

- LLM prompts, roles, RAG, tools, and agents
- Authentication versus authorization
- Least privilege, sandboxing, and trust boundaries
- Structured outputs and policy enforcement
- Red-team methodology

4. Core Concepts

| Concept | Meaning / why | Example | Interview angle |
|------------------------|---|--------------------------------|---|
| Direct injection | User explicitly overrides task | “Ignore previous instructions” | Input filtering alone is weak |
| Indirect injection | Malicious instruction in consumed content | Webpage says email secrets | Critical for RAG/agents |
| Instruction hierarchy | System/developer/user precedence | System policy over document | Model behavior, not hard access control |
| Data/control confusion | Untrusted text interpreted as commands | Retrieved note triggers tool | Core vulnerability |

| Concept | Meaning / why | Example | Interview angle |
|------------------------|----------------------------------|-----------------------------|-------------------------------------|
| Exfiltration | Attacker causes disclosure | Send private context to URL | Egress controls matter |
| Tool abuse | Unauthorized/harmful action | Delete files/send email | Least privilege + confirmation |
| Prompt leakage | Reveals hidden prompts/secrets | Echo system message | Never store real secrets in prompts |
| Capability confinement | Restricts reachable actions/data | Read-only scoped search | Most reliable mitigation layer |

5. Algorithm / Working Process

1. Classify every content source as trusted or untrusted and map privilege transitions.
2. Keep secrets out of prompts and separate tenants/authorization before retrieval.
3. Give the model narrowly scoped tools with typed arguments and minimum permissions.
4. Treat model output as an untrusted proposal; validate policy and authorization in deterministic code.
5. Require user confirmation for consequential actions and restrict network/file egress.
6. Delimit/label content and use injection detection as defense-in-depth.
7. Test direct, indirect, encoded, multilingual, multi-turn, and tool-chain variants; monitor abnormal actions.

6. Mathematical Foundation

Security aims to enforce an authorization invariant independent of model behavior:

$$Allowed(action, user, resource) = Policy(user, action, resource),$$

not $Allowed = LLM(prompt)$. If each of k independent opportunities has compromise probability p , system compromise can approach $1 - (1 - p)^k$; in practice failures are correlated, reinforcing the need for deterministic controls. Evaluate ASR and false-block rate under a stated attacker model.

7. Practical Implementation

```

from typing import Literal
from pydantic import BaseModel

class ReadRequest(BaseModel):
    action: Literal["read_document"]
    document_id: str

def execute(proposal: dict, user, acl, document_store):
    req = ReadRequest.model_validate(proposal) # model output is untrusted
    if not acl.can_read(user.id, req.document_id):
        raise PermissionError("Document access denied")
    return document_store.read(req.document_id)

```

8. Code Explanation

The model can propose only one typed read action. Deterministic code checks the actual user against the requested resource before access. The prompt cannot grant permission. Production code also rate-limits, logs decisions, avoids returning raw secrets, uses tenant-filtered retrieval, and isolates tool credentials.

9. Training / Evaluation

Build a matrix of source (user/document/web/tool/memory), encoding/language, objective (override/exfiltrate/action), and privilege. Measure attack success, unauthorized action/data rate, detection, false refusals, task utility, and action-confirmation bypass. Test the full orchestration with realistic identities and sandboxed tools.

10. Complexity and Cost

Schema/ACL checks are negligible. Additional classifiers and model calls add latency/cost but remain bypassable. Sandboxing, scoped credentials, confirmations, and egress proxies add engineering overhead while sharply limiting impact.

11. Common Use Cases

- RAG over uploaded or web documents
- Email/calendar/browser assistants
- Coding agents reading repositories/issues
- Customer-support tools accessing accounts
- Autonomous research and purchasing agents
- Multimodal systems reading text in images

12. Common Mistakes

- Asking the model to detect and ignore all attacks as the only defense
- Trusting retrieved content because it is in a company index
- Putting API keys/secrets in system prompts
- Giving a general shell/browser/database tool
- Checking authorization during retrieval but not tool execution—or vice versa
- Concatenating model output into commands/queries
- Auto-executing consequential actions
- Blocking literal phrases while missing encoded/indirect variants

13. Edge Cases / Limitations

Benign documents contain imperative language, making detection ambiguous. Images/encoded text and multi-step reasoning evade filters. Sometimes instructions inside documents are legitimately required. Models and attacks evolve, so no prompt-only defense guarantees separation. Even a perfect detector would not replace access control.

14. Variations

- **Direct/indirect injection:** user versus third-party content.

- **Cross-domain injection:** content from web/email controls another tool.
- **Stored injection:** persists in memory/database for later execution.
- **Multimodal injection:** instructions hidden in images/audio/layout.
- **Prompt leaking/exfiltration:** targets system text or connected data.
- **Tool-output injection:** compromised API response manipulates agent.

15. Related Topics

Jailbreaks seek policy bypass; prompt injection seeks instruction/control compromise, often for data/tool misuse. RAG expands the untrusted-input surface. Red teaming tests adaptive chains. PII handling reduces exposed data. Zero-trust authorization, sandboxing, and secure software design are the main defenses.

16. Interview Questions

1. **What is prompt injection?** Untrusted content manipulates an LLM's instructions or connected capabilities.
2. **Direct versus indirect?** User-supplied attack versus malicious third-party/retrieved content.
3. **Why not solve with system prompt?** Instruction precedence is probabilistic model behavior, not an authorization boundary.
4. **Strongest mitigation?** Reduce capability and enforce authorization/policy outside the model.
5. **Why not secrets in prompts?** Prompt context may be exposed through outputs, tools, logs, or injection.
6. **How secure tools?** Narrow typed APIs, scoped credentials, validation, ACLs, confirmations, sandboxing, audit.
7. **How secure RAG?** Tenant ACL before/after retrieval, provenance, content isolation, output checks, no autonomous privilege.
8. **Does detection solve it?** No; it is defense-in-depth with false positives/negatives.
9. **How evaluate?** End-to-end attack matrix with realistic privileges and unauthorized outcome metrics.
10. **Prompt injection versus SQL injection?** Similar data/control confusion; LLM lacks a reliable parameterization boundary, so contain authority.

17. Practice Tasks

- Threat-model a RAG email assistant.
- Replace a generic tool with two scoped typed operations.
- Create direct/indirect/multilingual regression cases.
- Find an authorization check performed only in the UI.
- Add confirmation and egress controls to a mock agent.

18. Project Ideas

| Project | What it does | Stack/dataset | Resume value |
|--------------------------|--|----------------------------------|---------------------------|
| Secure RAG Proxy | Enforces tenant ACL, provenance, and output policy | FastAPI, vector DB, Pydantic | Practical LLM security |
| Agent Capability Sandbox | Runs typed mock tools with scoped tokens | Python, containers/mock services | Secure agent architecture |

| Project | What it does | Stack/dataset | Resume value |
|-----------------------|--|-------------------------|---------------------|
| Injection Eval Matrix | Generates/tests indirect attack variants | promptfoo/garak, pytest | Red-team automation |

19. Quick Revision

- **Key idea:** untrusted data can become instructions; assume model compromise.
- **Formula:** deterministic `Allowed(user, action, resource)` invariant.
- **Use:** any LLM reading external content or using tools.
- **Metrics:** ASR, unauthorized data/action rate, false block, utility.
- **Trap:** prompt text is not a security boundary.
- **One-liner:** Let the model propose; let trusted code authorize and execute.

20. Final Cheat Sheet

| Item | Summary |
|-----------------|---|
| Definition | Instruction/control manipulation through untrusted content |
| Input/output | Malicious content + LLM system → override, disclosure, or action attempt |
| Steps | Map trust → minimize authority → typed proposals → ACL/policy → confirm → monitor |
| Hyperparameters | Tool scope, query/action limits, confirmation/detection thresholds |
| Metrics | ASR, unauthorized outcomes, detection, utility/false refusals |
| Pros/cons | Layered containment limits harm; prompt-level prevention remains imperfect |
| Best use | RAG, tool agents, browser/email/coding assistants |

Jailbreaks

1. Overview

A jailbreak is an adversarial interaction that causes a generative model to bypass intended safety, policy, or behavioral restrictions. It may use role-play, obfuscation, encoding, multilingual phrasing, long context, multi-turn escalation, model composition, or optimization. Jailbreak evaluation should measure harmful capability and policy violation under a defined policy, not merely the presence of disallowed keywords.

2. Intuition

If a safety-trained model is a helpful employee with rules, a jailbreak is social engineering designed to make it reinterpret, forget, or route around those rules. The durable defense is layered: model alignment, system controls, limited capabilities, monitoring, and safe product design.

3. Prerequisites

- LLM instruction tuning and preference optimization
- Safety policies and content taxonomies
- Adversarial prompting and red teaming
- Classifier evaluation and false positives/negatives
- Tool authorization and incident response

4. Core Concepts

| Concept | Meaning / why | Example category | Interview angle |
|---------------------|--|-------------------------------|---------------------------------------|
| Policy bypass | Produces behavior prohibited by scoped policy | Harmful procedural assistance | Need precise rubric |
| Universal jailbreak | Transfers across many prompts/models | Reusable suffix/pattern | Harder systemic risk |
| Obfuscation | Hides intent via encoding/language | Character substitution | Normalize but avoid brittle blacklist |
| Multi-turn attack | Builds benign context then escalates | Progressive elicitation | Evaluate conversation state |
| Refusal bypass | Model answers after framing manipulation | Fiction/role-play framing | Refusal style is not safety proof |
| Over-refusal | Benign request wrongly blocked | Safety education denied | Measure utility jointly |
| Adaptive attack | Optimizes against current defense | Search-generated suffix | Static test underestimates risk |
| Capability gating | Removes tools/data/actions despite text bypass | No dangerous tool access | Limits consequence |

5. Algorithm / Working Process

1. Define policy taxonomy, allowed transformations, severity, and evaluation rubric.
2. Build benign-neighbor and harmful test cases across languages and turns.
3. Run manual and automated adaptive attacks within authorized boundaries.
4. Judge outputs by whether they materially enable prohibited harm, not keyword match alone.
5. Measure ASR, severity, over-refusal, consistency, and end-to-end tool impact.
6. Mitigate through training, system prompting, classifiers, tool/data limits, rate limits, and review.
7. Retest adaptively and convert responsibly sanitized failures into regression cases.

6. Mathematical Foundation

Attack success under policy rubric J is

$$ASR = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[J(x_i, f(x_i)) = \textit{violation}].$$

Safety-utility tuning minimizes an expected cost such as

$$C_{harm}P(unsafe\ acceptance) + C_{block}P(benign\ refusal).$$

For stochastic models report success-at-least-once over k attempts, approximately $1 - (1 - p)^k$ only if trials are independent. State query budget and sampling parameters.

7. Practical Implementation

```
def evaluate_policy(cases, model_call, judge, attempts=3):
    rows = []
    for case in cases:
        outputs = [model_call(case["prompt"]) for _ in range(attempts)]
        verdicts = [judge(case["policy"], out) for out in outputs]
        rows.append({
            "id": case["id"], "harmful": case["harmful"],
            "any_violation": any(verdicts),
            "all_refused": all("refuse" in out.lower() for out in outputs),
        })
    return rows
```

8. Code Explanation

Multiple attempts expose stochastic bypasses. The independent `judge` applies an explicit policy rubric; substring refusal is recorded only as a crude behavior signal, not the safety verdict. A production harness randomizes ordering, stores redacted traces/version/config, uses human adjudication for severe ambiguity, and includes benign cases to measure over-refusal.

9. Training / Evaluation

Use frozen public-style attacks plus private adaptive cases and benign contrast sets. Report per-category/severity/language ASR, pass@attempt budget, harmfulness, refusal correctness, over-refusal, latency, and tool side effects. Validate automated judges against blinded experts and prevent the target model from being its sole judge.

10. Complexity and Cost

Cost grows with cases $\times$ variants $\times$ turns $\times$ attempts $\times$ judge calls. Optimization-based attacks can require many queries/gradients. Human expert review and safe handling of dangerous content dominate for high-risk domains.

11. Common Use Cases

- Public chat assistants and content generation
- Coding/cyber/bio-capable models
- Image/audio multimodal generation
- Child-facing or education products
- Agents with external tools

- Testing fine-tuned/quantized safety regressions

12. Common Mistakes

- Publishing actionable harmful outputs unnecessarily
- Counting any non-refusal as a successful harmful jailbreak
- Ignoring benign over-refusal and unequal language behavior
- Evaluating one deterministic attempt
- Using keyword filters as primary defense
- Patching exact strings without adaptive variants
- Testing the base model but not tools/system wrappers
- Vague policies that judges cannot apply consistently

13. Edge Cases / Limitations

Policies contain context-sensitive exceptions such as education, transformation, or defensive analysis. Automated judges share biases with targets. Attack taxonomies age quickly. Strong filtering can suppress legitimate discussion and disproportionately affect dialects/languages. No finite suite proves jailbreak resistance.

14. Variations

- **Role-play/obfuscation/multilingual:** manual prompt strategies.
- **Suffix/optimization attacks:** search gradients or model feedback.
- **Multi-turn/context attacks:** exploit conversation state.
- **Many-shot attacks:** overwhelm context with examples.
- **Multimodal jailbreaks:** hide instructions/requests across modalities.
- **Model-composition attacks:** use one model to transform or attack another.

15. Related Topics

Prompt injection compromises instruction/control boundaries, commonly to access data/tools; jailbreaks primarily bypass safety behavior. Red teaming covers both. Adversarial examples provide optimization concepts. Governance defines the policy and risk appetite; benchmark design ensures reproducible, non-contaminated evidence.

16. Interview Questions

1. **What is a jailbreak?** Adversarial elicitation that bypasses intended model safety or behavior constraints.
2. **Jailbreak versus prompt injection?** Policy bypass versus instruction/system compromise; attacks can overlap.
3. **What is ASR?** Fraction of attempts/cases producing policy-violating behavior under a rubric.
4. **Why test benign cases?** Safety controls can over-refuse legitimate requests.
5. **Why multiple attempts?** Sampling makes rare bypasses operationally relevant.
6. **Can filters solve jailbreaks?** No; use alignment plus policy enforcement, limited capabilities, monitoring, and adaptive testing.
7. **How judge success?** Material policy violation/harm enablement, not mere absence of refusal.

8. **What is a universal jailbreak?** A strategy/suffix that transfers across many prompts or models.
9. **How avoid benchmark overfitting?** Private/fresh adaptive sets and broad families, not exact strings.
10. **Most important agent defense?** Even if text safety fails, deterministic authorization and capability limits prevent harmful actions.

17. Practice Tasks

- Design harmful/benign contrast cases for a clear policy category.
- Measure ASR across temperatures and attempt budgets.
- Audit judge agreement with human labels.
- Diagnose over-refusal across languages.
- Turn a sanitized failure pattern into a family-based regression test.

18. Project Ideas

| Project | What it does | Stack/dataset | Resume value |
|---------------------------|---|-------------------------------------|------------------------------|
| Safety Eval Harness | Tracks ASR, over-refusal, judge agreement | HF, promptfoo/garak | LLM safety evaluation |
| Multilingual Safety Audit | Compares policy behavior across languages | multilingual LLM, custom safe cases | Inclusive safety research |
| Agent Consequence Gate | Shows capability limits surviving text bypass | Pydantic, mock tools, FastAPI | Defense-in-depth engineering |

19. Quick Revision

- **Key idea:** adversarially elicit behavior outside intended safety policy.
- **Formula:** rubric-based ASR and weighted harm/over-refusal cost.
- **Use:** generative-model safety testing and release gates.
- **Metrics:** ASR, severity, success@k, over-refusal, tool impact.
- **Trap:** non-refusal is not automatically unsafe; refusal is not complete defense.
- **One-liner:** Measure harmful outcomes and benign utility under adaptive attempts.

20. Final Cheat Sheet

| Item | Summary |
|-----------------|---|
| Definition | Adversarial bypass of safety/policy behavior |
| Input/output | Attack dialogue + model/system → policy-compliant or violating behavior |
| Steps | Define policy → build contrasts → attack → judge → mitigate → adaptive retest |
| Hyperparameters | Temperature, attempts, turns, query budget, classifier threshold |
| Metrics | ASR, severity, over-refusal, consistency, side effects |
| Pros/cons | Testing exposes gaps; suites age and judging is contextual |
| Best use | Public LLMs and capable tool-using/multimodal systems |

PII Handling

1. Overview

Personally identifiable information (PII) handling is the operational discipline of discovering, classifying, minimizing, protecting, using, sharing, retaining, and deleting data that identifies or can reasonably be linked to a person. In AI systems, PII may occur in structured fields, free text, images, audio, embeddings, prompts, retrieval indexes, traces, model outputs, checkpoints, and support tickets.

2. Intuition

PII is radioactive cargo: know where it came from, label it, move it through approved routes, expose it only to trained people/services, keep it only as long as justified, and verify disposal. Redaction at the UI is useless if raw prompts remain in logs and backups.

3. Prerequisites

- Privacy and security fundamentals
- Regex, NER, OCR, and data classification
- Encryption, key/secrets management, access control
- Data lineage, retention, deletion, and audit logging
- Threat modeling and incident response

4. Core Concepts

| Concept | Meaning / why | Example | Interview angle |
|-------------------------|--|----------------------------------|----------------------------|
| Direct identifier | Identifies a person alone | Email, government ID | Exact/validated detection |
| Quasi-identifier | Identifies through combination | ZIP + DOB + gender | Linkage risk |
| Sensitive PII | High-harm personal information | Health, biometrics, finance | Stronger controls |
| Detection | Find entities in all modalities | Regex + NER + OCR | Precision/recall trade-off |
| Redaction | Remove/mask value irreversibly for that copy | [EMAIL] | May destroy utility |
| Tokenization | Replace with reversible/random token | Vault-backed customer token | Keep mapping separately |
| Access/retention | Least privilege and limited lifetime | 30-day encrypted trace | Logs/backups count |
| Data subject operations | Access/correct/delete/export | Delete vector and derived record | Lineage is essential |

5. Algorithm / Working Process

1. Inventory every source/sink and classify direct, quasi, sensitive, confidential, and public fields.
2. Establish purpose and legal/organizational authority; reject unnecessary fields.
3. Detect PII at ingestion and before logging, external model calls, indexing, export, and output.
4. Redact, generalize, or tokenize according to downstream need.
5. Encrypt and enforce least privilege, tenant separation, audit trails, and scoped service identities.
6. Apply retention/deletion to primary data, caches, vectors, features, traces, backups, and artifacts.
7. Test detector recall on domain/language variants; monitor leaks and rehearse incident response.

6. Mathematical Foundation

PII detection is a classification/extraction problem. For entities,

$$Precision = \frac{TP}{TP + FP}, \quad Recall = \frac{TP}{TP + FN}, \quad F_\beta = (1 + \beta^2) \frac{PR}{\beta^2 P + R}.$$

High-risk egress often chooses $\beta > 1$ to emphasize recall, while excessive false positives can destroy document utility. Residual leak rate is leaked sensitive entities divided by actual sensitive entities. k-anonymity or risk scores can assess quasi-identifiers, but they do not replace access controls or formal DP.

7. Practical Implementation

```
import re

PATTERNS = {
    "email": re.compile(r"\b[A-Z0-9._%+-]+@[A-Z0-9.-]+\.[A-Z]{2,}\b", re.I),
    "phone": re.compile(r"(?! \d)(?:\+?\d[\s.-]?){9,14}\d(?:!\d)"),
}

def redact_pii(text: str) -> tuple[str, list[dict]]:
    findings = []
    for kind, pattern in PATTERNS.items():
        def replace(match):
            findings.append({"type": kind, "start": match.start(), "end":
match.end()})
            return f"[{kind.upper()}]"
        text = pattern.sub(replace, text)
    return text, findings

safe_text, audit = redact_pii("Contact Ana at ana@example.com or +91 98765 43210")
print(safe_text, audit)
```

8. Code Explanation

The example replaces common emails/phone-like strings while recording only type and offset, not secret value. Regex is useful for structured patterns but has false positives and misses names, addresses, context, multilingual forms, OCR errors, and novel IDs. Production pipelines combine validators, NER/OCR, field metadata, allow/deny policies, and reviewed exceptions before any untrusted sink.

9. Training / Evaluation

Build a versioned gold set representing formats, languages, noise, and domain entities. Report entity- and exact/span-level precision/recall by type, severity-weighted leakage, throughput, and utility after transformation. Include adversarial separators/Unicode/OCR and non-PII lookalikes. Evaluate every boundary, not just ingestion. Re-test after detector/model updates.

10. Complexity and Cost

Regex scanning is approximately $O(LK)$ for text length L and patterns K ; optimized engines reduce practical cost. NER/OCR adds model inference and GPUs may help at scale. Token vaults, encryption, lineage, deletion, and access reviews add persistent operational cost.

11. Common Use Cases

- Scrubbing prompts before external LLM APIs
- Customer-support transcript analytics
- Medical NLP and document ingestion
- RAG indexing and retrieval authorization
- Observability/logging for AI agents
- Dataset publishing and annotation platforms

12. Common Mistakes

- Using regex alone for all PII
- Detecting at ingestion but leaking in logs/errors/outputs
- Storing raw detected values in audit records
- Assuming names are always PII and IDs are never PII
- Replacing with stable plain hashes
- Sharing embeddings or screenshots without scanning
- Deleting source rows but not indexes/features/backups/models
- Sending PII to third parties without purpose, terms, or controls

13. Edge Cases / Limitations

PII is contextual and jurisdiction-dependent. Common names may not identify anyone; rare combinations may. Free text, code, medical terminology, images, audio, and multilingual scripts are difficult. Perfect recall is impossible, and over-redaction harms task performance. Seek privacy/legal expertise for real policy obligations.

14. Variations

- **Masking/redaction:** irreversible per released copy; analytics/logging.
- **Tokenization/pseudonymization:** reversible through secure vault; record linkage.
- **Generalization:** DOB to age band; reduces specificity.
- **Format-preserving encryption:** legacy field compatibility; careful cryptography required.
- **NER/OCR/multimodal detectors:** unstructured text/images/audio.
- **DLP gateways:** organization-wide policy at egress/storage boundaries.

15. Related Topics

Privacy supplies the broader purpose/use framework. Differential privacy protects releases/models from individual influence but does not permit careless raw PII handling. RAG security requires authorization in addition to redaction. Dataset cards document sensitive fields and consent; governance defines retention and incident owners.

16. Interview Questions

1. **What is PII?** Information that identifies or can reasonably be linked to a person, directly or in combination.
2. **Direct versus quasi-identifier?** Direct works alone; quasi-identifiers link through combinations/external data.
3. **Redaction versus tokenization?** Remove/mask versus replace with a controlled linkable token.
4. **Why not plain hashing?** Predictable values can be dictionary-reversed and stable hashes enable linkage.
5. **How select detector threshold?** From harm/cost: high-risk egress often prioritizes recall, then measures utility loss.
6. **Where scan?** Ingestion, logs, external calls, indexes, tool outputs, exports, and generated responses.
7. **Are embeddings PII-free?** No; they can retain/link sensitive semantics and may be inverted.
8. **How delete from RAG?** Remove source, chunks, vectors, caches, replicas, and audit completion by lineage.
9. **How handle logs?** Minimize/redact before writing, restrict access, encrypt, expire, and audit.
10. **Is PII handling only regex?** No; it is end-to-end governance plus technical detection/protection/deletion.

17. Practice Tasks

- Extend the detector with validators and false-positive tests.
- Build a multilingual gold set and plot precision–recall.
- Trace PII through a RAG pipeline and design deletion.
- Find leaks in exception traces and observability payloads.
- Implement random tokenization with a mocked secure vault.

18. Project Ideas

| Project | What it does | Stack/dataset | Resume value |
|------------------------|---|---------------------------------|--------------------------------|
| LLM Privacy Gateway | Detects/redacts before prompts/logs and scans outputs | Presidio, spaCy, FastAPI | Production privacy engineering |
| RAG Deletion Auditor | Verifies source-to-vector/cache deletion | FAISS, SQLite, Python | Data lineage and compliance |
| Multimodal PII Scanner | OCR + NER + metadata detection | OpenCV, Tesseract, transformers | CV/NLP security |

19. Quick Revision

- **Key idea:** control PII at every lifecycle boundary, not only source tables.
- **Formula:** type/slice precision, recall, F_β , residual leak rate.

- **Use:** personal-data ingestion, AI calls, RAG, logs, releases.
- **Metrics:** recall by severity/type, false redaction, leak rate, deletion SLA.
- **Trap:** pseudonymized or embedded data can still be personal.
- **One-liner:** Minimize first, then detect, transform, authorize, retain briefly, and verify deletion.

20. Final Cheat Sheet

| Item | Summary |
|-----------------|---|
| Definition | Lifecycle controls for identifiable/linkable information |
| Input/output | Raw multimodal records → minimized/redacted/tokenized controlled data |
| Steps | Inventory → classify → minimize → detect → protect → delete → audit |
| Hyperparameters | Detector threshold, retention, access scope, token policy |
| Metrics | Per-type precision/recall, leaks, utility loss, deletion latency |
| Pros/cons | Reduces exposure; detection is imperfect and controls add overhead |
| Best use | Logs, RAG, support/health/finance data, external AI APIs |

Formal Verification

1. Overview

Formal verification uses mathematical specifications and proof/search techniques to establish that a system satisfies a property under explicit assumptions. In AI it can verify neural-network robustness within a bounded region, controller safety, monotonicity, output bounds, fairness constraints, protocol logic, and software surrounding models. It provides stronger evidence than testing but only for the formalized model, property, and environment.

2. Intuition

Testing checks many chosen points; verification attempts to prove no point in a defined region violates the rule. It is the difference between testing several bridge loads and proving, from a mathematical model, that every load up to a bound respects a stress limit.

3. Prerequisites

- Logic, predicates, quantifiers, invariants
- Linear algebra, optimization, and neural networks
- SAT/SMT, linear/mixed-integer programming basics
- State machines and reachability
- Floating-point and implementation semantics

4. Core Concepts

| Concept | Meaning / why | Example | Interview angle |
|-------------------------|---|--------------------------------------|---|
| Specification | Formal property to prove/refute | Output class unchanged within radius | Most failures start with wrong spec |
| Soundness | “Verified” claims are trustworthy under assumptions | Sound over-approximation | Completeness may be sacrificed |
| Counterexample | Concrete input violating property | Perturbation causing unsafe class | Validate on real implementation |
| SAT/SMT | Solves logical formulas with theories | ReLU/linear constraints | Exact but can scale poorly |
| MILP | Encodes piecewise-linear networks | Binary ReLU activation variables | Complete for bounded problem |
| Abstract interpretation | Over-approximates reachable values | Interval/polytope bounds | Scalable but conservative |
| Reachability | All states reachable under dynamics | Robot remains outside obstacle | Core control-system property |
| Runtime assurance | Checks/enforces invariants online | Safety shield overrides policy | Useful when offline proof is incomplete |

5. Algorithm / Working Process

1. Define system boundary, input domain, environment assumptions, and safety property.
2. Translate model/controller/software and property into solver-supported constraints.
3. Choose complete exact solving or sound conservative relaxation based on scale/risk.
4. Ask whether the negation of the property is satisfiable.
5. If SAT, validate and minimize counterexample; fix system/spec.
6. If UNSAT under sound encoding, record the scoped proof/certificate and tool versions.
7. Check implementation equivalence and monitor assumptions at runtime.

6. Mathematical Foundation

For local classification robustness around x_0 , verify

$$\forall x : \|x - x_0\|_\infty \leq \epsilon, \text{quad} f_y(x) > f_j(x) \forall j \neq y.$$

The solver searches the negation: $\exists x$ within the box and some $j \neq y$ with $f_j(x) \geq f_y(x)$. A ReLU $z = \max(0, a)$ can be MILP-encoded with bounds $L \leq a \leq U$ and a binary activation variable. For control, an invariant $I(s)$ is inductive if it holds initially and $I(s) \wedge T(s, s') \Rightarrow I(s')$.

7. Practical Implementation

```
from z3 import Real, Solver, And, Or, sat
```

```
# Verify: for x in [0, 1], ReLU(2x - 0.5) never exceeds 1.5.
x, a, z = Real("x"), Real("a"), Real("z")
s = Solver()
s.add(And(x >= 0, x <= 1), a == 2*x - 0.5)
s.add(Or(And(a >= 0, z == a), And(a <= 0, z == 0)))
s.add(z > 1.5) # Negation of the desired property.
result = s.check()
assert result != sat, f"Counterexample: {s.model()}"
print("Verified for the encoded real-arithmetic model and input interval")
```

8. Code Explanation

The solver receives the input domain, exact small ReLU relation, and negated output bound. `unsat` means no violating real-valued input exists in this encoding. It does not automatically cover floating-point inference, preprocessing, quantization, a different checkpoint, or inputs outside `[0,1]`; those assumptions belong in the certificate/model card.

9. Training / Evaluation

Verification complements—not replaces—test performance. Report verified property rate across representative inputs, certified radius, solver timeout/unknown rate, proof method/soundness, and counterexamples. Train with verifiable objectives or simpler/constrained architectures if proof coverage matters. Validate counterexamples end-to-end and test numerical equivalence.

10. Complexity and Cost

Exact ReLU verification is generally NP-complete and worst-case exponential in nonlinear units. MILP/SAT may time out on large networks; abstract methods scale better but return inconclusive bounds. Verification is usually CPU/memory intensive, though bound propagation can use GPUs.

11. Common Use Cases

- Collision avoidance and autonomous control
- Neural-network local robustness certification
- Monotonic/output-bound constraints in risk models
- Safety shields for reinforcement learning
- Protocol, authorization, and agent workflow invariants
- Quantized/embedded neural systems

12. Common Mistakes

- Verifying a weak/wrong property and claiming system safety
- Omitting preprocessing, postprocessing, tools, or environment
- Confusing timeout/unknown with verified
- Ignoring floating-point/quantization mismatch
- Claiming global robustness from local bounded proofs
- Using an unsound approximation without disclosure
- Failing to validate solver counterexamples in deployed code
- Not versioning the exact model and verifier

13. Edge Cases / Limitations

Open-world language behavior is hard to specify formally. Environment models may omit rare reality. Specifications can conflict or encode harmful assumptions. Large transformers and stochastic systems are difficult. Proofs can be invalidated by compiler, hardware, preprocessing, model, or configuration changes.

14. Variations

- **SMT/MILP exact verification:** small piecewise-linear networks; strong but costly.
- **Abstract interpretation/interval bound propagation:** scalable sound over-approximation.
- **Randomized smoothing:** probabilistic robustness certificate under noise.
- **Model checking:** state-transition systems and policies.
- **Theorem proving:** high-assurance mathematical/software proofs.
- **Runtime verification/shields:** enforce temporal/safety rules online.

15. Related Topics

Adversarial testing finds counterexamples but cannot prove absence; verification can prove a bounded property. Robustness supplies common specifications. Interpretability can simplify verification. RL safety uses reachability and shields. Governance determines which properties and proof coverage satisfy release requirements.

16. Interview Questions

1. **What is formal verification?** Mathematical proof/search that a system satisfies a specification under explicit assumptions.
2. **Verification versus testing?** Testing samples cases; verification quantifies over a defined set.
3. **What does UNSAT mean?** No counterexample exists in the encoded model/domain if the encoding/tool is sound.
4. **What does SAT mean?** A candidate counterexample satisfies the negated property.
5. **What is sound but incomplete?** Never falsely verifies, but may return unknown/fail to verify true properties.
6. **Why are ReLU networks hard?** Combinatorial activation patterns make exact search worst-case exponential.
7. **What is abstract interpretation?** Soundly over-approximate sets of internal values to bound all behaviors.
8. **What is a robustness certificate?** Proof prediction is invariant within specified norm/radius/assumptions.
9. **Biggest practical risk?** Specification/implementation mismatch.
10. **Can we verify an LLM is truthful?** Not generally; narrow structured components/properties may be verified, not open-world truthfulness.

17. Practice Tasks

- Encode and verify bounds for a two-layer ReLU network.
- Generate and validate a counterexample for a monotonicity property.
- Compare interval bounds with exact MILP on small networks.
- Find floating-point behavior differing from real arithmetic.

- Design a runtime shield for a grid-world RL policy.

18. Project Ideas

| Project | What it does | Stack/dataset | Resume value |
|----------------------------|---|----------------------------------|-----------------------------|
| Neural Robustness Verifier | Certifies small MNIST networks at radii | PyTorch, auto_LiRPA/Marabou | Formal ML research |
| Safe RL Shield | Blocks unsafe actions using invariants | Gymnasium, Z3 | RL + formal methods |
| Monotonic Credit Proof | Verifies direction/output constraints | ONNX, SMT/MILP; synthetic credit | Regulated high-assurance ML |

19. Quick Revision

- **Key idea:** prove no counterexample exists inside an explicit formal scope.
- **Formula:** $\forall x \in B(x_0, \epsilon) : f_y(x) > f_j(x)$.
- **Use:** bounded high-assurance properties and safety invariants.
- **Metrics:** verified rate/radius, timeout/unknown, counterexamples, runtime.
- **Trap:** a proof is only as good as specification, encoding, and implementation match.
- **One-liner:** Verification replaces “we tested many cases” with “none exist”—but only inside the modeled boundary.

20. Final Cheat Sheet

| Item | Summary |
|-----------------|--|
| Definition | Proof of formal properties under explicit assumptions |
| Input/output | System + domain + property $\rightarrow$ proof/UNSAT, counterexample/SAT, or unknown |
| Steps | Specify $\rightarrow$ encode $\rightarrow$ negate $\rightarrow$ solve $\rightarrow$ validate $\rightarrow$ certify/monitor |
| Hyperparameters | Input bounds, radius/norm, timeout, relaxation precision |
| Metrics | Certified radius/rate, solve time, unknown rate, counterexamples |
| Pros/cons | Strong scoped guarantee; hard to specify/scale and vulnerable to mismatch |
| Best use | Safety controllers, local robustness, monotonicity, protocol invariants |

Differential Privacy

1. Overview

Differential privacy (DP) is a formal guarantee that a randomized computation's output changes only slightly when one individual's data is added or removed. It limits membership and contribution disclosure even against attackers with auxiliary information. DP is used for aggregate statistics, telemetry, synthetic data, and privacy-preserving model training.

2. Intuition

If a report/model looks almost statistically the same whether you participate or not, observing it reveals little about your individual record. Noise is calibrated to the query's sensitivity, and repeated releases consume a finite privacy budget.

3. Prerequisites

- Probability distributions, randomness, and logarithms
- Query sensitivity and norms
- SGD, per-example gradients, clipping
- Composition and sampling
- Privacy threat models and utility metrics

4. Core Concepts

| Concept | Meaning / why | Example | Interview angle |
|----------------------|--|--------------------------------------|--|
| Neighboring datasets | Differ by one person's contribution | Add/remove one user | User-level vs record-level matters |
| ϵ | Multiplicative privacy loss | Smaller is stronger | Meaning depends on mechanism/composition |
| δ | Small probability of relaxed failure | Much smaller than population inverse | Not "probability data leaks" exactly |
| Sensitivity | Max query change between neighbors | Count sensitivity 1 | Calibrates noise |
| Composition | Privacy losses accumulate | Many training steps/releases | Track accountant |
| Post-processing | Any data-independent processing preserves DP | Publish model predictions | No extra privacy cost |
| Amplification | Random subsampling strengthens privacy | Minibatch sampling | Accountant uses sampling rate |
| DP-SGD | Clip per-example gradients and add noise | Private neural training | Key implementation |

5. Algorithm / Working Process

For DP-SGD: define privacy unit and target budget; sample a minibatch; compute each example/user gradient; clip each gradient to norm C ; sum clipped gradients; add Gaussian noise proportional to σC ; update parameters; and use a privacy accountant over sampling rate and steps. Stop if the budget exceeds the approved limit, then evaluate utility and privacy assumptions.

6. Mathematical Foundation

A randomized mechanism M is (ϵ, δ) -DP if for all neighboring D, D' and events S ,

$$P[M(D) \in S] \leq e^\epsilon P[M(D') \in S] + \delta.$$

Laplace mechanism releases $f(D) + \text{Lap}(\Delta_1 f / \epsilon)$, where $\Delta_1 f = \max_{D \sim D'} \|f(D) - f(D')\|_1$. Gaussian noise supports approximate DP. DP-SGD clips

$$\bar{g}_i = g_i \min(1, C / \|g_i\|_2), \quad \tilde{g} = \frac{1}{B} \left(\sum_i \bar{g}_i + \mathcal{N}(0, \sigma^2 C^2 I) \right).$$

Tighter composition uses Rényi DP or privacy-loss accountants rather than naively adding epsilons.

7. Practical Implementation

```
import numpy as np

def dp_count(bits, epsilon, rng=None):
    """One bounded contribution per person; returns an epsilon-DP noisy count."""
    if epsilon <= 0:
        raise ValueError("epsilon must be positive")
    bits = np.asarray(bits)
    if not np.isin(bits, [0, 1]).all():
        raise ValueError("each person must contribute 0 or 1")
    rng = rng or np.random.default_rng()
    sensitivity = 1.0
    return bits.sum() + rng.laplace(0.0, sensitivity / epsilon)

print(dp_count([1, 0, 1, 1], epsilon=1.0, rng=np.random.default_rng(42)))
```

8. Code Explanation

Bounding one binary contribution gives count sensitivity 1. Laplace scale is sensitivity divided by epsilon. A seed is used only to reproduce the example; production randomness must be appropriately secure and seeds protected. Multiple releases compose, and unbounded repeated queries can average away noise, so a budget/accounting service is required.

9. Training / Evaluation

Report privacy unit, adjacency, ϵ , δ , clipping norm, noise multiplier, sampling rate, steps, accountant, and utility. Evaluate overall/slice accuracy, calibration, convergence, and membership attacks as sanity evidence (attacks do not replace the proof). Tune on public/non-sensitive or budget-accounted validation; hyperparameter searches can consume privacy.

10. Complexity and Cost

Simple DP queries are cheap. DP-SGD needs per-example gradients, clipping, accounting, and often larger batches/more training; memory/runtime can be several times standard SGD. Stronger privacy generally reduces utility, especially with small data, rare groups, and large models.

11. Common Use Cases

- Government/statistical aggregate releases
- Product telemetry and federated analytics
- Private image/text/tabular model training
- Language-model fine-tuning on sensitive text
- Synthetic data generation with formal training protection
- Cross-organization research outputs

12. Common Mistakes

- Adding arbitrary noise without sensitivity analysis
- Reporting epsilon without adjacency, delta, accountant, or composition
- Treating delta as ordinary failure probability
- Releasing repeated noisy answers without budget control
- Clipping batch gradients rather than per-example/user gradients
- Claiming DP because membership attacks failed
- Ignoring preprocessing/hyperparameter selection on private data
- Using record-level DP when one person contributes many correlated records

13. Edge Cases / Limitations

DP does not make false/biased data safe, prevent authorized misuse, hide population facts, or protect data before the mechanism. Outliers and rare groups may lose disproportionate utility. Large epsilon gives weak protection; there is no universal “good epsilon.” Correlated individuals and user contribution bounds complicate semantics.

14. Variations

- **Central DP:** trusted curator adds noise; best utility.
- **Local DP:** users randomize before sharing; stronger trust model, more noise.
- **Shuffle DP:** anonymizing shuffle amplifies local privacy.
- **DP-SGD:** private model training; placement/research important.
- **User-level DP:** protects all records of a person; operationally meaningful.
- **PATE:** aggregates noisy teacher votes; useful when public unlabeled data exists.

15. Related Topics

Privacy gives the lifecycle objective; DP provides a formal release guarantee. Federated learning changes where computation occurs and often combines with user-level DP/secure aggregation. PII handling is still required before/around DP. Formal verification and accounting validate narrow mathematical claims. Fairness analysis is needed because DP noise can hurt small groups.

16. Interview Questions

1. **Define differential privacy.** Neighboring datasets induce nearly indistinguishable output distributions bounded by ϵ, δ .
2. **Smaller epsilon means?** Stronger privacy, usually less utility.
3. **What is sensitivity?** Maximum query change when one privacy unit changes.
4. **Laplace mechanism?** Add Laplace noise scaled to L1 sensitivity divided by epsilon.

5. **What is composition?** Privacy loss accumulates across releases/steps.
6. **Post-processing property?** Processing a DP output without private data cannot worsen its DP guarantee.
7. **How DP-SGD works?** Per-example/user gradient clipping, Gaussian noise, and accounting.
8. **Why clipping?** Bounds each privacy unit's influence/sensitivity.
9. **Record versus user DP?** One row versus all contributions from one person.
10. **Does DP guarantee model accuracy/fairness?** No; evaluate utility and group effects separately.

17. Practice Tasks

- Implement DP count/mean with bounded contribution and composition.
- Plot noise/error versus epsilon and dataset size.
- Train MNIST with Opacus and report epsilon–accuracy trade-off.
- Debug incorrect batch-level clipping.
- Compare record- and user-level contribution bounds.

18. Project Ideas

| Project | What it does | Stack/dataset | Resume value |
|-----------------------------|---|--------------------------|-------------------------------|
| DP Analytics Service | Budgets and releases private counts/histograms | NumPy, FastAPI, OpenDP | Formal privacy engineering |
| DP-SGD Benchmark | Compares utility, calibration, membership attacks | PyTorch, Opacus; MNIST | ML privacy research |
| Private Telemetry Simulator | Models local/shuffle/central DP trade-offs | Python; synthetic events | Systems/privacy understanding |

19. Quick Revision

- **Key idea:** bound how much any one privacy unit changes output distribution.
- **Formula:** $P[M(D) \in S] \leq e^\epsilon P[M(D') \in S] + \delta$.
- **Use:** aggregate releases and sensitive-data model training.
- **Metrics:** ϵ , δ , utility, clipping/noise, privacy attack sanity tests.
- **Trap:** arbitrary noise or failed attacks are not DP.
- **One-liner:** Bound contribution, calibrate noise to sensitivity, and account every release.

20. Final Cheat Sheet

| Item | Summary |
|-----------------|--|
| Definition | Formal neighboring-dataset output-indistinguishability guarantee |
| Input/output | Bounded private data + randomized mechanism $\rightarrow$ DP statistic/model |
| Steps | Define unit $\rightarrow$ bound/clip $\rightarrow$ add calibrated noise $\rightarrow$ account $\rightarrow$ evaluate |
| Hyperparameters | ϵ , δ , C , σ , sampling rate, steps |
| Metrics | Privacy budget and task/slice utility |

| Item | Summary |
|-----------|--|
| Pros/cons | Robust formal privacy; utility/compute cost and scoped assumptions |
| Best use | Telemetry, statistics, private training, federated deployments |

Federated Learning

1. Overview

Federated learning (FL) trains a shared model across decentralized clients while keeping raw training records local. Clients compute updates and a coordinator aggregates them. FL is used across phones, hospitals, banks, vehicles, and organizations where data movement is costly, regulated, private, or impossible. FL reduces centralization but does not by itself guarantee privacy or security.

2. Intuition

Instead of bringing every notebook to one office, send a draft to participants; each edits using local experience, and the coordinator combines edits. The edits can still leak information or be malicious, and participants' data/devices are non-identical.

3. Prerequisites

- SGD and distributed optimization
- Weighted averaging and sampling
- Non-IID data and domain shift
- Differential privacy and secure aggregation
- Distributed systems, failures, and threat models

4. Core Concepts

| Concept | Meaning / why | Example | Interview angle |
|--------------------|--|-------------------------------|----------------------------------|
| Cross-device FL | Huge unreliable device population | Mobile keyboard | Partial participation, privacy |
| Cross-silo FL | Few reliable organizations | Hospitals | Governance/schema heterogeneity |
| Non-IID data | Client distributions differ | Each user has unique language | Causes client drift |
| FedAvg | Local SGD then weighted update average | Weight by local examples | Standard algorithm |
| Secure aggregation | Server learns only aggregate | Mask individual updates | Dropout-tolerant protocol needed |
| Client selection | Sample eligible clients per round | Online/charging phones | Selection bias |

| Concept | Meaning / why | Example | Interview angle |
|---------------------|--------------------------------------|------------------------|-----------------------------------|
| Poisoning/backdoors | Malicious clients alter global model | Triggered update | Robust aggregation and monitoring |
| Personalization | Adapt shared model locally | User-specific keyboard | Global vs local utility |

5. Algorithm / Working Process

1. Server initializes global parameters w_0 .
2. Each round samples eligible clients under participation/privacy rules.
3. Server sends current model/config.
4. Each client trains locally for E epochs on its private data and returns an update.
5. Secure aggregation/validation combines allowed updates, often weighted by local sample count.
6. Server updates global model and evaluates central/federated metrics.
7. Repeat until convergence/budget, then optionally personalize; monitor drift, attacks, fairness, and dropouts.

6. Mathematical Foundation

Global objective across clients k is

$$F(w) = \sum_{k=1}^K \frac{n_k}{n} F_k(w), \quad F_k(w) = \frac{1}{n_k} \sum_{i \in D_k} \ell(w; x_i, y_i).$$

FedAvg aggregates selected client models:

$$w_{t+1} = \sum_{k \in S_t} \frac{n_k}{\sum_{j \in S_t} n_j} w_{t+1}^k.$$

Multiple local steps reduce communication but under heterogeneous F_k cause client drift. FedProx adds $\frac{\mu}{2} \|w - w_t\|^2$ to local objectives. Robust aggregators use coordinate median/trimmed mean under assumptions about malicious-client fraction.

7. Practical Implementation

```
import torch

def fedavg(client_states, client_sizes):
    total = sum(client_sizes)
    if total <= 0 or len(client_states) != len(client_sizes):
        raise ValueError("valid aligned clients and sizes required")
    merged = {}
    for name in client_states[0]:
        merged[name] = sum(
            state[name].detach() * (size / total)
            for state, size in zip(client_states, client_sizes)
        )
    return merged
```

```
# global_model.load_state_dict(fedavg(local_state_dicts, local_sample_counts))
```

8. Code Explanation

The server averages each parameter tensor weighted by client examples, matching the empirical global objective. Detaching prevents autograd graphs from crossing rounds. Real FL additionally validates shape/version, handles buffers and sparse updates, uses secure aggregation, clips updates, tolerates dropout, and avoids trusting self-reported sample counts blindly.

9. Training / Evaluation

Split/evaluate by clients and time. Report global weighted and unweighted client metrics, percentiles/worst-client, convergence rounds, communication bytes, participation/dropout, calibration, privacy budget, and attack robustness. Use simulations with realistic non-IID partitions, then small controlled deployments. Keep a server-side representative set only if governance permits.

10. Complexity and Cost

Client compute is local epochs $\times$ local examples; server aggregation is linear in participating clients and parameter count. Communication often dominates: roughly two model-sized transfers per participating client/round. Compression and more local epochs save bandwidth but can harm convergence. Secure aggregation and DP add protocol/utility overhead.

11. Common Use Cases

- On-device keyboard, speech, and personalization
- Multi-hospital medical models
- Cross-bank fraud intelligence
- Vehicle/fleet perception and telemetry
- Industrial IoT and edge analytics
- Privacy-preserving recommendation

12. Common Mistakes

- Claiming raw-data locality equals privacy
- Assuming client data are IID
- Evaluating only average global accuracy
- Trusting arbitrary client updates/sample counts
- Ignoring stragglers, dropout, availability, battery, and bandwidth
- Using secure aggregation as poisoning defense
- Applying record-level DP when user-level protection is needed
- Centralizing verbose client telemetry that recreates privacy risk

13. Edge Cases / Limitations

Clients can be offline, slow, malicious, or highly imbalanced. Rare client populations may be underselected. Gradient/update leakage remains. Secure aggregation hides individual updates from server inspection,

complicating anomaly detection. Cross-silo parties need schema alignment and legal agreements. FL can cost more than approved centralized learning.

14. Variations

- **FedAvg**: baseline; placement-essential.
- **FedProx/SCAFFOLD**: reduce non-IID client drift.
- **Personalized FL**: local heads/fine-tuning/meta-learning.
- **Federated analytics**: aggregate statistics without training.
- **Split learning**: divide model computation across client/server.
- **Vertical FL**: parties hold different features for overlapping entities.
- **Secure/DP FL**: combine aggregation cryptography with user-level noise.

15. Related Topics

Differential privacy bounds update/model leakage; secure aggregation cryptographically hides individual updates. Robustness covers poisoning and non-IID shift. Privacy/PII handling still governs devices, logs, and consent. Governance coordinates participating organizations, ownership, deletion, and incident response.

16. Interview Questions

1. **What is federated learning?** Distributed training where raw client records stay local and updates are aggregated.
2. **How does FedAvg work?** Clients perform local SGD; server sample-count-weights their resulting models/updates.
3. **Why non-IID is hard?** Local objectives point in different directions, slowing/destabilizing convergence.
4. **Does FL ensure privacy?** No; updates may leak and server/clients may be malicious.
5. **What is secure aggregation?** Protocol revealing only aggregate updates, not each client's update.
6. **Cross-device versus cross-silo?** Many unreliable devices versus few reliable organizations.
7. **How handle client drift?** Fewer local steps, proximal/control-variate methods, better sampling, personalization.
8. **How defend poisoning?** Authentication, clipping, robust aggregation, anomaly monitoring, validation, constrained capabilities.
9. **What metrics beyond accuracy?** Client percentiles/fairness, rounds, bytes, dropout, privacy, attack resistance.
10. **When not use FL?** If central use is permissible/cheaper and distributed complexity adds no material benefit.

17. Practice Tasks

- Simulate FedAvg on IID versus label-skewed MNIST clients.
- Plot accuracy versus local epochs/communication rounds.
- Compare weighted average and client-macro performance.
- Inject a malicious update and test clipping/median aggregation.
- Add user-level DP accounting to a federated simulation.

18. Project Ideas

| Project | What it does | Stack/dataset | Resume value |
|--------------------------|---------------------------------------|--|-----------------------------------|
| Non-IID FL Simulator | Compares FedAvg/FedProx/SCAFFOLD | Flower, PyTorch; FEMNIST | Distributed ML research |
| Federated Hospital Model | Cross-silo training with site metrics | Flower, sklearn; partitioned clinical data | Privacy-aware health ML |
| Poison-Resistant FL Lab | Tests Byzantine updates and defenses | PyTorch; CIFAR-10 | ML security + distributed systems |

19. Quick Revision

- **Key idea:** move model computation/updates, not raw client datasets.
- **Formula:** sample-weighted local objective and FedAvg.
- **Use:** data cannot/should not be centralized and benefit outweighs complexity.
- **Metrics:** client/global utility, rounds, bytes, dropout, privacy, robustness.
- **Trap:** FL is an architecture, not a privacy guarantee.
- **One-liner:** FedAvg is distributed local SGD; non-IID data, leakage, attacks, and communication are the real engineering problems.

20. Final Cheat Sheet

| Item | Summary |
|-----------------|---|
| Definition | Collaborative decentralized model training without moving raw data |
| Input/output | Global model + local datasets → aggregated global/personalized models |
| Steps | Select → distribute → local train → protect/aggregate → update → evaluate |
| Hyperparameters | Clients/round, local epochs, LR, clipping, noise, proximal μ |
| Metrics | Utility distribution, rounds/bytes, dropout, privacy budget, ASR |
| Pros/cons | Data locality/personalization; complex, non-IID, leaky, attackable |
| Best use | Cross-device/cross-silo settings with genuine locality constraints |

AI Governance

1. Overview

AI governance is the system of decision rights, policies, processes, evidence, roles, and technical controls used to direct and oversee AI across its lifecycle. It turns principles such as fairness, privacy, safety, transparency, security, and accountability into inventories, risk tiers, approval gates, ownership, monitoring, incident response, and retirement. Governance covers the full socio-technical system, including vendors, data, users, and downstream effects.

2. Intuition

Governance is air-traffic control for AI: it does not design every aircraft, but establishes who may fly, required checks, routes, logs, emergency procedures, and authority to ground an unsafe system.

3. Prerequisites

- ML/MLOps lifecycle and system architecture
- Risk management and internal controls
- Responsible AI, privacy, security, and evaluation
- Product ownership and incident management
- Basic awareness that laws/standards vary and evolve by jurisdiction

4. Core Concepts

| Concept | Meaning / why | Example | Interview angle |
|------------------------|---|--|-----------------------------------|
| AI inventory | Registry of systems, owners, uses, versions, dependencies | All deployed/vendor models | You cannot govern unknown systems |
| Risk classification | Proportional controls based on impact/exposure | Low-risk autocomplete vs clinical triage | Risk-based, not one-size-fits-all |
| Accountability/RACI | Named responsible/approving/consulted roles | Product owner accepts residual risk | Committees do not replace owners |
| Lifecycle gates | Evidence required before stages | Privacy/security/eval before release | Automate objective gates |
| Policies/standards | Required outcomes and implementation rules | Human review for adverse decisions | Avoid vague principles |
| Traceability | Link purpose, data, code, model, evaluation, approval, deployment | Registry and signed run | Reproduce decisions |
| Monitoring/incidents | Detect harm/drift and respond | Rollback on critical threshold | Include user complaints |
| Third-party governance | Assess vendors/models/data | API terms, eval, change notice | Responsibility is not outsourced |

5. Algorithm / Working Process

1. Inventory the proposed system, use, stakeholders, data, vendors, jurisdictions, and decision impact.
2. Classify risk using severity, likelihood, scale, reversibility, autonomy, and vulnerable populations.
3. Assign accountable owner and required specialist reviews/controls.
4. Translate requirements into measurable acceptance criteria and evidence artifacts.

5. Run data/model/system evaluations, red teams, privacy/security reviews, and human-factors assessment.
6. Approve, conditionally approve, or reject with recorded residual risk and expiry.
7. Monitor performance, incidents, complaints, changes, and vendor updates; re-review material changes and retire safely.

6. Mathematical Foundation

Governance uses decision analysis rather than one universal loss. Expected harm can be approximated by

$$EH = \sum_s P(s) \text{Impact}(s) \text{Exposure}(s),$$

then modified by control effectiveness and uncertainty. A release gate may require several conjunctive criteria:

$$\text{Release} = (\text{Utility} \geq u_0) \wedge (\text{RiskMetrics} \leq r_0) \wedge (\text{EvidenceComplete}) \wedge (\text{OwnerApproved}).$$

For rare catastrophic harms, expected value alone is insufficient; apply hard constraints, worst-case analysis, precaution, and human escalation. Quantitative scores support—not replace—judgment.

7. Practical Implementation

```
REQUIRED_BY_RISK = {
    "low": {"owner", "model_card"},
    "medium": {"owner", "model_card", "privacy_review", "monitoring_plan"},
    "high": {"owner", "model_card", "dataset_card", "privacy_review",
            "security_review", "red_team", "human_oversight", "rollback_plan"},
}

def release_decision(system: dict) -> tuple[bool, list[str]]:
    risk = system["risk_tier"]
    missing = sorted(REQUIRED_BY_RISK[risk] - set(system.get("evidence", [])))
    failed = [name for name, passed in system.get("release_gates", {}).items() if
not passed]
    reasons = [f"missing:{x}" for x in missing] + [f"failed:{x}" for x in failed]
    return not reasons, reasons
```

8. Code Explanation

The code applies proportionate evidence requirements and fails closed when required evidence or a gate is missing. It is workflow enforcement, not the risk assessment itself. Production governance needs immutable artifact links, approver identities, separation of duties, exceptions with expiry, vendor/change triggers, audit logs, and emergency rollback.

9. Training / Evaluation

Governance evaluates both AI systems and governance effectiveness. System gates include utility, calibration, fairness, privacy, security, robustness, human oversight, latency, and cost. Program metrics include inventory coverage, evidence freshness, exceptions, time-to-review, incidents, near misses, time-to-detect/contain,

overdue remediation, rollback tests, and complaint outcomes. Avoid rewarding teams merely for document volume.

10. Complexity and Cost

Cost grows with risk, autonomy, novelty, scale, sensitive data, and jurisdictions. Automation handles inventory, lineage, tests, expiry, and evidence collection; expert and stakeholder judgment remains necessary. Excessive bureaucracy drives shadow AI, so controls should be proportional and integrated into existing engineering workflows.

11. Common Use Cases

- Enterprise model/GenAI approval and registry
- High-impact hiring, credit, health, education, insurance
- Vendor foundation-model and dataset procurement
- Agent/tool autonomy decisions
- Regulatory/audit evidence management
- Incident response, change management, and retirement

12. Common Mistakes

- Publishing principles without owners or enforceable gates
- Treating governance as legal/compliance team's job alone
- Governing a model artifact but ignoring workflow/users/tools/vendors
- One checklist for every risk tier
- Approving once and never monitoring changes
- Allowing undocumented permanent exceptions
- Using a numeric risk score to hide uncertainty/judgment
- Counting documents instead of measuring risk outcomes

13. Edge Cases / Limitations

Rules and standards differ and change by jurisdiction; this guide is not legal advice. Novel systems may not fit taxonomies. Too little governance permits harm; too much creates delay and evasion. Vendor opacity limits evidence. Some harms appear only after broad deployment, requiring staged rollout, monitoring, user recourse, and stop authority.

14. Variations

- **Centralized governance board:** consistent oversight; can bottleneck.
- **Federated governance:** domain teams own controls under central standards; scales with assurance.
- **Three-lines model:** business ownership, risk oversight, independent audit.
- **Risk-tiered lifecycle:** controls proportional to impact; placement/industry essential.
- **Continuous controls monitoring:** automated evidence and drift/change triggers.
- **Regulatory/standards mappings:** map one control to multiple obligations; maintain current authoritative sources.

15. Related Topics

Model and dataset cards provide evidence; benchmarks, fairness, robustness, privacy, verification, and red teaming test requirements. MLOps supplies lineage, reproducibility, monitoring, rollback, and access controls. Human-centered design supplies notice, consent, contestability, accessibility, and recourse. Security governance integrates threat and incident management.

16. Interview Questions

1. **What is AI governance?** Decision rights, lifecycle processes, evidence, and controls that keep AI aligned with organizational and societal requirements.
2. **Why inventory first?** Unknown systems have no owner, risk tier, monitoring, or retirement path.
3. **What is risk-tiered governance?** Stronger controls for higher impact, exposure, autonomy, or uncertainty.
4. **Who owns model risk?** A named accountable business/product owner, supported by technical/risk specialists.
5. **What belongs in a release gate?** Reproducible utility and risk evidence, ownership, monitoring, oversight, rollback, and resolved failures.
6. **How govern third-party models?** Contract/use review, independent evaluation, data terms, change monitoring, fallback, incident obligations.
7. **What triggers re-review?** Material model/data/prompt/tool/use/population/vendor/regulatory change or incident/drift.
8. **How avoid checkbox governance?** Measurable outcomes, artifact linkage, technical enforcement, audits, incident learning, user feedback.
9. **What is residual risk?** Risk remaining after controls, explicitly accepted by authorized owner for a scoped period.
10. **How measure governance success?** Coverage/freshness plus fewer/severer incidents, rapid detection/remediation, effective rollback and recourse—not paperwork count.

17. Practice Tasks

- Build a risk-tier questionnaire for three example AI systems.
- Define release gates and evidence for a hiring model and a writing assistant.
- Create a RACI and material-change policy.
- Run a tabletop AI incident and identify missing telemetry/authority.
- Design metrics that detect both weak controls and excessive approval friction.

18. Project Ideas

| Project | What it does | Stack/dataset | Resume value |
|------------------------|--|----------------------------|----------------------------|
| AI Governance Registry | Inventories systems, risks, evidence, owners, expiry | FastAPI, Postgres, React | Enterprise AI platform |
| Policy-as-Code Gates | Blocks deployment on missing/failed evidence | Python, MLflow, CI/CD | MLOps + governance |
| AI Incident Simulator | Tabletop scenarios, timelines, roles, lessons | Streamlit; synthetic cases | Risk/operations leadership |

19. Quick Revision

- **Key idea:** convert AI principles and obligations into owned, evidence-based lifecycle decisions.
- **Formula:** expected harm plus hard constraints; conjunctive release gate.
- **Use:** all organizational AI, proportionate to risk.
- **Metrics:** inventory/evidence coverage, risk gates, incidents, detection/remediation, exceptions.
- **Trap:** governance is not a checklist or one-time approval.
- **One-liner:** If a system has no owner, evidence, monitor, rollback, and re-review trigger, it is not governed.

20. Final Cheat Sheet

| Item | Summary |
|-----------------|--|
| Definition | Organizational decision rights, evidence, processes, and controls for AI |
| Input/output | AI use/system + risks + obligations → tier, controls, approval, monitoring |
| Steps | Inventory → classify → own → assess/test → decide → monitor → re-review/retire |
| Hyperparameters | Risk thresholds, evidence expiry, control gates, exception duration |
| Metrics | Coverage, freshness, failed gates, incidents, response, remediation, friction |
| Pros/cons | Accountability and consistent risk control; can become stale/bureaucratic |
| Best use | Enterprise portfolios, high-impact systems, vendors, agents, regulated domains |